

On-Policy Self-Distillation for Reasoning Compression

Hejian Sang*

hejian@alumni.iastate.edu

Yuanda Xu*

yuanda@math.princeton.edu

Zhengze Zhou*

zz433@cornell.edu

Ran He*

rh2528@columbia.edu

Zhipeng Wang

zhipeng.wang@alumni.rice.edu

Jiachen Sun

jiachens@umich.edu

Abstract

Reasoning models think out loud, but much of what they say is noise. We introduce OPSDC (**O**n-**P**olicy **S**elf-**D**istillation for Reasoning **C**ompression), a method that teaches models to reason more concisely by distilling their own concise behavior back into themselves. The entire approach reduces to one idea: condition the same model on a “be concise” instruction to obtain teacher logits, and minimize per-token reverse KL on the student’s own rollouts. No ground-truth answers, no token budgets, no difficulty estimators. Just self-distillation. Yet this simplicity belies surprising sophistication: OPSDC automatically compresses easy problems aggressively while preserving the deliberation needed for hard ones. On Qwen3-8B and Qwen3-14B, we achieve 57–59% token reduction on MATH-500 while *improving* accuracy by 9–16 points absolute. On AIME 2024, the 14B model gains 10 points with 41% compression. The secret? Much of what reasoning models produce is not just redundant—it is actively harmful, compounding errors with every unnecessary token. Code is available at https://github.com/HJSang/OPSD_Reasoning_Compression.

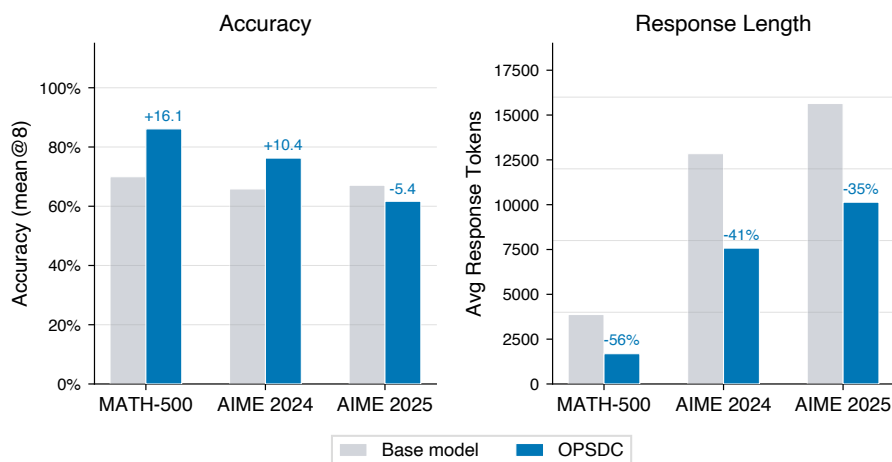


Figure 1: **The paradox of reasoning compression: less thinking, better answers.** Results for Qwen3-14B across three benchmarks of increasing difficulty (30K response token budget). OPSDC compresses reasoning traces by 35–57% while largely preserving or *improving* accuracy, most dramatically on MATH-500, where accuracy jumps from 70.0% to 86.1%.

*Equal contribution.

†Correspondence to hejian@alumni.iastate.edu

1 Introduction

Modern reasoning models have learned to think before they speak, and they have a lot to say. Systems like OpenAI o1 (Jaech et al., 2024), Gemini 2.5 (Comanici et al., 2025), DeepSeek-R1 (Guo et al., 2025), and Qwen3 (Yang et al., 2025) produce thousands of tokens of internal deliberation before arriving at an answer, exploring blind alleys, second-guessing themselves, and verifying conclusions. This verbosity pays off on hard problems. But it comes at a cost: these models *cannot stop talking*, even when the answer is obvious. Ask them what $2 + 2$ is, and they may spend 500 tokens considering whether you meant binary arithmetic (Snell et al., 2024; Muennighoff et al., 2025).

The community has noticed. A flurry of compression methods has emerged (Appendix B surveys some recent approaches), each attacking the problem from a different angle. But every existing paradigm demands a sacrifice: RL methods need ground-truth answers and risk collapsing the model’s ability to explore (Aggarwal and Welleck, 2025; Wan et al., 2026; Liu et al., 2025); SFT methods train on someone else’s reasoning and forget their own (Huang et al., 2025; Shenfeld et al., 2026); most treat all problems alike, compressing a trivial sum as aggressively as a competition integral; and prompting tricks vanish the moment you remove the prompt.

We propose OPSDC (**O**n-**P**olicy **S**elf-**D**istillation for Reasoning **C**ompression), a method that sidesteps all of these trade-offs with a single, almost trivial idea: *ask the model to be concise, then teach it to do so without being asked*. The model already knows how to compress; it just needs permission. We give it that permission via a conciseness instruction, then distill this behavior back into the base model. No rewards, no budgets, no oracles. Given a reasoning model π_θ , we define:

- **Teacher:** $\pi_\theta(\cdot \mid x, c)$, the same model conditioned on a conciseness instruction c (for example: “Solve concisely, avoid unnecessary steps”).
- **Student:** $\pi_\theta(\cdot \mid x)$, the same model without the compression instruction.

Training generates student rollouts and minimizes the per-token reverse KL divergence between student and teacher distributions. This on-policy self-distillation approach requires no ground-truth answers, no reward engineering, and no difficulty estimation. The compression signal emerges naturally from the KL objective, adapting automatically to problem difficulty.

Table 1: **Comparison of reasoning compression methods.** OPSDC uniquely combines on-policy training, no dependence on ground-truth (GT) answers, difficulty-adaptive compression, and entropy preservation.

Method	On-policy	No GT needed	Difficulty-adaptive	Entropy-preserving
RL + length penalty (Aggarwal and Welleck, 2025; Wan et al., 2026)	✓	✗	✗	✗
SFT on compressed CoT (Huang et al., 2025)	✗	✗	✗	✓
OPCD (Ye et al., 2026)	✓	✗	✗	✓
DLER (Liu et al., 2025)	✓	✗	✗	✗
Prompting / pruning (Xu et al., 2025)	—	✓	✗	✓
OPSDC (ours)	✓	✓	✓	✓

Table 1 contrasts OPSDC with representative methods from each paradigm. OPSDC is the only approach that satisfies all four desiderata.

Summary of results. On Qwen3-8B and Qwen3-14B, OPSDC achieves **57–59% token reduction on MATH-500 while improving accuracy by 9–16 percentage points** (to $\sim 86\%$). On AIME 2024, the 14B model gains 10 points with 41% compression. Compression naturally adapts to difficulty ($\sim 1.6\times$ more compression on easy vs. hard problems), entropy remains stable throughout training, and general capabilities (MMLU) are fully preserved.

2 Related Work

Reasoning compression via reinforcement learning. The most direct approach: penalize length in the reward function. L1 (Aggarwal and Welleck, 2025) caps token count during GRPO training.

DiPO (Wan et al., 2026) and DIET (Chen et al., 2025) estimate difficulty from rollout pass rates and set per-problem length targets. Leash (Li et al., 2025b) shapes rewards with sigmoid functions; DLER (Liu et al., 2025) adds curriculum learning. ThinkPrune (Hou et al., 2025) continuously trains long-thinking LLMs using reinforcement learning (RL) with an additional token-budget constraint while preserving answer correctness. The catch: all of these require ground-truth answers. No correct answer, no reward and no way to know if compression went too far. Xu et al. (2026) further notes that reinforcement learning can induce overconfidence errors, thereby narrowing the model’s reasoning boundary and reducing generation diversity.

Reasoning compression via supervised fine-tuning. Another route: curate short reasoning traces, then train on them. SEER (Huang et al., 2025) samples many solutions and keeps the shortest correct ones. TokenSkip (Xia et al., 2025) learns which tokens to skip. DAP/LiteCoT (Wu et al., 2025) distills from stronger models; S3-CoT (Du et al., 2026) steers activations toward brevity. The problem is distribution shift: the student trains on someone else’s reasoning and forgets its own (Shenfeld et al., 2026).

Training-free compression. The lightweight option: change the prompt or the decoder, not the weights. Chain of Draft (Xu et al., 2025) asks for minimal drafts instead of full reasoning. TrimR (Lin et al., 2025) prunes after the fact. NoWait (Wang et al., 2025a) and FlowSteer (Li et al., 2026) steer decoding toward conciseness. These methods are easy to deploy but achieve limited compression, and the effect vanishes when you change the prompt.

On-policy self-distillation. The closest relatives of our work use the model as its own teacher. OPSD (Zhao et al., 2026) gives the teacher the ground-truth answer, achieving $4\text{--}8\times$ efficiency over GRPO. SDPO (Hübotter et al., 2026) conditions on rich feedback for dense credit assignment. SDFT (Shenfeld et al., 2026) shows that on-policy distillation dramatically reduces forgetting compared to standard SFT, interpreting it as inverse RL. OPCD (Ye et al., 2026) distills system-prompt behaviors into weights. We contribute a new application: using a *conciseness instruction* as the privileged context, achieving compression without any ground-truth supervision.

3 Method

3.1 Problem Formulation

Consider a reasoning model π_θ that, given input x , generates a reasoning trace r followed by an answer a , producing output $y = (r, a)$. The reasoning trace typically appears within `<think>...</think>` delimiters. We aim to learn parameters θ^* such that the model produces shorter reasoning traces while maintaining accuracy.

Let c denote a conciseness instruction (see Figure 2 for a concrete example). Modern reasoning models can follow such instructions via in-context learning, producing shorter reasoning traces when c is prepended to the input. We denote the conciseness-conditioned model as $\pi_\theta(\cdot \mid x, c)$ (teacher) and the unconditional model as $\pi_\theta(\cdot \mid x)$ (student). The teacher and student share parameters θ but receive different inputs.

3.2 Training Objective

OPSDC minimizes the per-token reverse KL divergence between the student and a stop-gradient teacher on student-generated rollouts:

$$\mathcal{L}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot \mid x)} \left[\sum_{t=1}^{|y|} D_{\text{KL}} \left(\pi_\theta(\cdot \mid x, y_{<t}) \parallel \pi_{\bar{\theta}}(\cdot \mid x, c, y_{<t}) \right) \right], \quad (1)$$

where $\bar{\theta}$ denotes the teacher weights, which are periodically synchronized with the student (Section 3.3), and no gradients flow through the teacher’s forward pass. The expectation over $y \sim \pi_\theta(\cdot \mid x)$ makes training *on-policy*: the student is optimized on its own generation distribution, which prevents the distribution shift inherent in off-policy SFT.

Student Prompt $\pi_{\theta}(\cdot \mid x)$
Solve the following math problem step by step. The last line of your response should be of the form Answer: \$Answer (without quotes) where \$Answer is the answer to the problem. Find all real numbers x such that $x^3 - 6x^2 + 11x - 6 = 0$. Remember to put your answer on its own line after “Answer:”.
Teacher Prompt $\pi_{\bar{\theta}}(\cdot \mid x, c)$
Conciseness instruction c: Solve the following math problem concisely and correctly. Be direct -- avoid unnecessary elaboration, redundant steps, or restating the problem. Focus only on the key reasoning steps needed to reach the answer. The last line of your response should be of the form Answer: \$Answer (without quotes) where \$Answer is the answer to the problem. Find all real numbers x such that $x^3 - 6x^2 + 11x - 6 = 0$. Remember to put your answer on its own line after “Answer:”.

Figure 2: **Prompt example for student and teacher policies.** Both policies share the same model parameters but differ in conditioning context. The teacher receives only a *conciseness instruction* c prepended to the problem; no ground-truth answers or reference solutions are provided. This is the key distinction from prior self-distillation work (Shenfeld et al., 2026), where the teacher receives the ground-truth solution as privileged information. The student prompt is the original prompt from the DAPO-17K dataset.

Why reverse KL? The choice of divergence direction matters critically in the iterative setting. Reverse KL ($D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\bar{\theta}})$) weights each gradient update by the *student’s own* distribution: the student only adjusts in token regions it currently generates, providing natural self-regularization against the periodic teacher refreshes. Forward KL ($D_{\text{KL}}(\pi_{\bar{\theta}} \parallel \pi_{\theta})$) weights updates by the *teacher’s* distribution instead, producing unconstrained gradient signals whose magnitude is independent of how far the student has drifted. In practice (Appendix G), forward KL causes progressive accuracy collapse synchronized with every teacher refresh—a saw-tooth that deepens with each cycle, reaching a $> 23\%$ AIME gap by step 190 on Qwen3-14B—alongside an aggressive compression of response lengths that truncates the reasoning chains needed for hard problems. Reverse KL is immune to this pathology: because the student already covers the teacher’s high-probability modes, each refresh requires only a small incremental adjustment, and accuracy remains stable throughout training.

3.3 Teacher Parameterization

A natural baseline is a *fully frozen* teacher ($\bar{\theta} = \theta_0$) as in Zhao et al. (2026). While simple and stable, the frozen teacher becomes an increasingly weak compression target as the student improves: once the student has internalized the initial conciseness signal, no further compression is possible because the reference distribution no longer leads the student.

To address this, we adopt a *periodic teacher update* strategy. The teacher weights are synchronized with the current student weights every M training steps:

$$\bar{\theta} \leftarrow \theta \quad \text{every } M \text{ steps.} \quad (2)$$

Each refresh creates a new, stronger compression target: the updated teacher, when conditioned on the conciseness instruction c , produces traces that are more concise than the previous teacher’s (since the student, now serving as the new teacher, has already learned to compress). This *progressive compression* effect pushes the student to continuously shorten its reasoning over the course of training, beyond what a single frozen reference can achieve.

Algorithm 1: OPSDC: On-Policy Self-Distillation for Concise Reasoning

Input: Model π_θ , dataset $\mathcal{D} = \{x_i\}$, conciseness instruction c , learning rate η , teacher update interval M

Output: Compressed reasoning model π_{θ^*}

Initialize teacher: $\bar{\theta} \leftarrow \theta_0$;

for each training step $k = 1, 2, \dots$ **do**

if $k \bmod M = 0$ **then**

 Update teacher: $\bar{\theta} \leftarrow \theta$; // periodic refresh

end

 Sample batch $\{x_1, \dots, x_B\} \sim \mathcal{D}$;

for each x_i in batch **do**

 Generate student rollout: $y_i \sim \pi_\theta(\cdot | x_i)$;

for each token position $t = 1, \dots, |y_i|$ **do**

 Compute student logits: $q_t \leftarrow \pi_\theta(\cdot | x_i, y_{i, < t})$;

 Compute teacher logits: $p_t \leftarrow \pi_{\bar{\theta}}(\cdot | x_i, c, y_{i, < t})$; // no grad

 Compute $D_{\text{KL}}(q_t \| p_t)$;

end

$\mathcal{L}_i \leftarrow \sum_t D_{\text{KL}}(q_t \| p_t)$;

end

 Update student: $\theta \leftarrow \theta - \eta \nabla_\theta \frac{1}{B} \sum_i \mathcal{L}_i$; // normalized by $|y_i|$ in practice

 ;

end

return π_{θ^*} ;

Difficulty-adaptive compression. Compression adapts naturally to problem difficulty: for easy problems, the concise teacher produces much shorter traces, creating strong KL signal; for hard problems, even the teacher needs extensive reasoning, yielding weak signal. We formalize this in Proposition 1 and verify it empirically in Section 5.3.

3.4 Training Algorithm

The complete OPSDC training procedure is given in Algorithm 1.

Computational cost and simplicity. The entire training pipeline requires only standard supervised training infrastructure: no reward models, no value functions, no advantage estimation, and no multi-rollout sampling. Each training step requires two forward passes per rollout token: one for the student (with gradient) and one for the teacher (without gradient, and cacheable within each M -step window). The periodic teacher refresh (Eq. 2) is a simple weight copy with negligible cost. This simplicity yields substantial efficiency gains over RL methods, which require multiple rollouts per prompt, reward model inference, and complex optimization (e.g., PPO clipping, GAE).

4 Theoretical Analysis

We now summarize key theoretical properties of OPSDC that illuminate why such a simple objective can produce strong compression without the failure modes of length-penalized RL. In particular, we connect the per-token loss to sequence-level KL, interpret the update as implicit reward maximization, and analyze when compression preserves accuracy, adapts to difficulty, and avoids catastrophic forgetting. Proof sketches are provided inline; full proofs are deferred to Appendix A.

4.1 Training Loss as Sequence-Level KL

The first result connects the practical per-token training objective to a standard information-theoretic quantity, enabling all subsequent analysis.

Lemma [Training loss equals sequence-level KL; Lemma 1]. The per-token OPSDC loss (Eq. 1) equals the sequence-level KL divergence $D_{\text{KL}}(\pi_\theta(\cdot | x) \parallel \pi_{\bar{\theta}}(\cdot | x, c))$ by the chain rule for autoregressive models.

Proof sketch. By the autoregressive factorization $q(y | x) = \prod_t q(y_t | x, y_{<t})$, the log-ratio $\log \frac{q(y|x)}{p(y|x)}$ decomposes into $\sum_t \log \frac{q(y_t|x, y_{<t})}{p(y_t|x, y_{<t})}$. Taking expectations over $y \sim q$ yields the per-token KL sum, which equals the sequence-level KL by definition. $\square$

This identification underpins all subsequent results by letting us apply standard information-theoretic tools (Pinsker’s inequality, the data-processing inequality) to the per-token loss.

4.2 Implicit Reward Interpretation

Following the inverse RL framework of Shenfeld et al. (2026), we show that OPSDC implicitly maximizes a reward function that combines task performance with a conciseness preference.

Theorem 1 (Implicit reward). *The OPSDC objective (Eq. 1) is equivalent to maximizing the expected implicit reward:*

$$r(y_t, x) = \log \pi_{\bar{\theta}}(y_t | x, c, y_{<t}) - \log \pi_\theta(y_t | x, y_{<t}). \quad (3)$$

Proof sketch. Expanding the reverse KL:

$$D_{\text{KL}}(\pi_\theta(\cdot | x, y_{<t}) \parallel \pi_{\bar{\theta}}(\cdot | x, c, y_{<t})) = \mathbb{E}_{y_t \sim \pi_\theta} \left[\log \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_{\bar{\theta}}(y_t | x, c, y_{<t})} \right] \quad (4)$$

$$= -\mathbb{E}_{y_t \sim \pi_\theta} [r(y_t, x)]. \quad (5)$$

Since $r(y_t, x) = \log \pi_{\bar{\theta}} - \log \pi_\theta$ naturally decomposes into a teacher-favoring term $\log \pi_{\bar{\theta}}$ and an entropy-like term $-\log \pi_\theta$, minimizing reverse KL can be interpreted as maximizing an implicit, policy-dependent reward-shaping objective. This is closely related to maximum-entropy RL, but not identical to the standard setting with a fixed environment reward. $\square$

Remark 1. *The implicit reward $r(y_t, x)$ is positive when the concise teacher assigns higher probability to token y_t than the student does, and negative otherwise. Thus, OPSDC implicitly rewards concise reasoning without any explicit length penalty. Within each M -step window, the teacher weights θ are fixed, serving as an implicit trust region: the student can only move as far as the teacher’s concise distribution allows, preventing unbounded policy drift. The periodic refresh (Eq. 2) then shifts this trust region forward, enabling progressive compression across windows.*

4.3 Accuracy Preservation

A natural concern is whether compression degrades accuracy. The following theorem shows that accuracy loss is bounded by two interpretable quantities.

Theorem [Accuracy preservation; Theorem 2]. If training converges to loss ϵ_{KL} and the concise teacher preserves accuracy to within ϵ_T of the base model, the student satisfies:

$$\text{Acc}(\pi_{\theta^*}) \geq \text{Acc}(\pi_{\bar{\theta}}) - \epsilon_T - \sqrt{\epsilon_{\text{KL}}/2}. \quad (6)$$

Proof sketch. Apply Pinsker’s inequality to convert the KL bound ϵ_{KL} into a total variation bound $\sqrt{\epsilon_{\text{KL}}/2}$ between student and teacher distributions. Since total variation bounds the difference in probability of any event—in particular, the correctness event $A(x)$ —the student’s accuracy is within $\sqrt{\epsilon_{\text{KL}}/2}$ of the teacher’s. Combining with the teacher quality assumption ϵ_T via the triangle inequality yields the result. $\square$

The bound decomposes accuracy loss into two independent, interpretable terms: teacher quality (ϵ_T) and distillation gap ($\sqrt{\epsilon_{\text{KL}}/2}$). In practice, ϵ_T is *negative* (the concise teacher is more accurate than the base model), which is why compression *improves* accuracy. The bound becomes $\text{Acc}(\pi_{\theta^*}) \geq$

$\text{Acc}(\pi_{\bar{\theta}}) + |\epsilon_T| - \sqrt{\epsilon_{\text{KL}}/2}$: accuracy improves whenever the teacher’s gain exceeds the distillation gap.

4.4 Difficulty-Adaptive Compression

A key design question for any compression method is how to allocate budget across problems of varying difficulty. We show that OPSDC handles this *automatically*: the compression signal is provably stronger on easy problems.

Proposition [Difficulty-adaptive compression; Proposition 1]. The per-token compression signal $S(x)$ is non-increasing in problem difficulty $d(x)$: easy problems receive strong compression pressure while hard problems, whose reasoning steps are predominantly essential, receive weak pressure.

Proof sketch. Decompose the normalized KL into essential and compressible token contributions: $S(x) = \rho(x) \cdot D_{\mathcal{E}} + (1 - \rho(x)) \cdot D_{\mathcal{C}}$, where $\rho(x)$ is the fraction of essential tokens, and $D_{\mathcal{E}}, D_{\mathcal{C}}$ are the category-level KL divergences. Since compressible tokens carry strictly larger KL ($D_{\mathcal{C}} > D_{\mathcal{E}}$) and the essential fraction $\rho(x)$ is non-decreasing in difficulty, $S(x) = D_{\mathcal{C}} - \rho(x)(D_{\mathcal{C}} - D_{\mathcal{E}})$ is a decreasing affine function of $\rho(x)$, hence non-increasing in $d(x)$. $\square$

This formalizes the empirical pattern (Table 2) that OPSDC compresses aggressively on MATH-500 ($\sim 57\text{--}59\%$) but conservatively on AIME ($\sim 35\%$), without any explicit difficulty estimation.

4.5 Bounded Forgetting

A central advantage of on-policy self-distillation over off-policy SFT is controlled divergence from the original model. We formalize this via the *conciseness gap*.

Proposition [Bounded forgetting; Proposition 2]. Divergence from the base model is bounded by:

$$\mathbb{E}_x[d_{\text{TV}}(\pi_{\theta^*}(\cdot | x), \pi_{\theta_0}(\cdot | x))] \leq \sqrt{\epsilon_{\text{KL}}/2} + \mathbb{E}_x[\gamma(x)], \quad (7)$$

where $\gamma(x)$ is the conciseness gap, i.e., the total variation between the base model’s outputs with and without the conciseness instruction.

Proof sketch. Apply the triangle inequality for total variation: $d_{\text{TV}}(\pi_{\theta^*}, \pi_{\theta_0}) \leq d_{\text{TV}}(\pi_{\theta^*}, \pi_{\theta_0}(\cdot | c)) + d_{\text{TV}}(\pi_{\theta_0}(\cdot | c), \pi_{\theta_0})$. The first term is bounded by $\sqrt{\epsilon_{\text{KL}}/2}$ via Pinsker’s inequality on the converged training loss; the second term is the conciseness gap $\gamma(x)$ by definition. $\square$

For hard problems where the conciseness instruction has little effect, $\gamma(x) \approx 0$, so forgetting is minimal precisely where it matters most. This contrasts with off-policy SFT, whose forgetting depends on the full distribution mismatch between teacher data and the base model—a gap that can be arbitrarily large.

4.6 Compression Reduces Compounding Error

Finally, we provide a probabilistic model explaining the most striking empirical finding: shorter reasoning traces can *improve* accuracy rather than degrade it.

Proposition [Compression reduces compounding error; Proposition 3]. Under a model where each token independently introduces a reasoning error with probability p_{err} , compressing from L to αL tokens yields an accuracy ratio $(1 - p_{\text{err}})^{\alpha L} / (1 - p_{\text{err}})^L = (1 - p_{\text{err}})^{-(1-\alpha)L}$, which grows exponentially in the number of removed tokens.

Proof sketch. Direct computation: the accuracy ratio $(1 - p_{\text{err}})^{\alpha L} / (1 - p_{\text{err}})^L = (1 - p_{\text{err}})^{-(1-\alpha)L}$. Using $\ln(1 - p) \leq -p$ and $e^u \geq 1 + u$, this is at least $1 + (1 - \alpha)L \cdot p_{\text{err}}$. On MATH-500 with

Table 2: **Self-distillation compresses reasoning traces while improving accuracy without forgetting (token budget = 30K).** Results on Qwen3-8B and Qwen3-14B with a 30,000-token budget to eliminate truncation effects. Accuracy (Acc, mean over 8 samples per problem, %), average reasoning token length (Len), and token reduction relative to the base model (Red., %). “Concise prompt” uses the conciseness instruction at inference only (no training); OPSDC trains with periodic teacher update ($M=50$). The rightmost column reports MMLU (Hendrycks et al., 2020) accuracy to verify that general capabilities are preserved. Results under the efficient-serving budget (8,192 tokens) are in Table 5 (Appendix C.1).

Method	MATH-500			AIME 2024			AIME 2025			MMLU
	Acc	Len	Red.	Acc	Len	Red.	Acc	Len	Red.	Acc
<i>Qwen3-8B</i>										
Base Model	77.7	4,661	—	72.5	14,170	—	62.5	16,682	—	73.2
Concise prompt	80.9	2,941	36.9%	67.5	11,589	18.2%	56.3	14,347	14.0%	—
OPSDC	86.6	1,921	58.8%	69.6	9,152	35.4%	57.1	10,726	35.7%	73.3
<i>Qwen3-14B</i>										
Base Model	70.0	3,872	—	65.8	12,844	—	67.1	15,642	—	76.9
Concise prompt	83.8	2,426	37.3%	68.3	9,866	23.2%	58.3	12,831	18.0%	—
OPSDC	86.1	1,686	56.5%	76.3	7,577	41.0%	61.7	10,137	35.2%	76.9

$L \approx 4,660$ and $\alpha \approx 0.41$ (Qwen3-8B, 30K budget), even $p_{\text{err}} = 10^{-4}$ yields a $\sim 28\%$ relative accuracy improvement. $\square$

This provides a *lower* bound on the accuracy benefit of compression: in practice, reasoning errors are positively correlated (one incorrect step causes subsequent steps to build on a false premise), amplifying the gain beyond the independence assumption. This explains why our empirical accuracy improvements (e.g., 70.0 $\rightarrow$ 86.1 on MATH-500) substantially exceed what the simple independent-error model predicts.

5 Experiments

5.1 Experimental Setting

Models and data. We evaluate OPSDC on Qwen3-8B and Qwen3-14B (Yang et al., 2025), training on $\sim 13,600$ competition-level math problems from DAPO-Math-17k (Yu et al., 2025) *without ground-truth answers*; only problem statements are used to generate student rollouts. We train for 1 epoch with learning rate 1×10^{-6} , global batch size 32, periodic teacher update (interval $M=50$; see ablation in Section 5.7.2), and $8 \times$ H200 GPUs. Although nominally a full epoch, the algorithm converges quickly at around ~ 100 steps. Each prompt generates a single student rollout (temperature 1.0) with a maximum response length of 8,192 tokens. Because OPSDC optimizes a per-token KL objective rather than an outcome-based reward, there is no need to generate complete responses as in RL methods; partial rollouts already provide a useful training signal. This phenomenon is also observed by Chen et al. (2025b) in the context of knowledge distillation. Full training and infrastructure details are in Appendix D.

Benchmarks. We evaluate on three mathematical reasoning benchmarks spanning a wide difficulty range: MATH-500 (Hendrycks et al., 2021) (500 problems, base accuracy 70–78%), AIME 2024 (30 problems, 66–73%), and AIME 2025 (30 problems, 63–67%).³ We define a *token budget* as the maximum response length allowed during inference, a practical lever for controlling serving cost. We report results under two budgets: 8,192 tokens, representative of efficient serving constraints, and 30,000 tokens, which effectively eliminates truncation and enables fairer accuracy comparison.

5.2 Self-Distillation Simultaneously Compresses and Improves Reasoning

Table 2 presents our main results under the 30,000-token budget, which eliminates response truncation for a fair accuracy comparison.⁴

Finding 1: Less is more. OPSDC simultaneously improves accuracy *and* reduces reasoning length on MATH-500, the opposite of the trade-off everyone assumes. Both models reach ~86% accuracy (from 77.7% for 8B and 70.0% for 14B) while shedding **57–59%** of their tokens. On AIME 2024, the 14B model improves substantially (65.8→76.3, +10.5pp) with 41% compression. On the harder AIME 2025, accuracy decreases modestly (~5pp) as the model trades some deliberation for efficiency.

Notably, the concise prompt alone (no training) already improves MATH-500 accuracy while reducing tokens by 14–37%, confirming that much of the base model’s reasoning is redundant. OPSDC with periodic teacher update ($M=50$) amplifies both effects, removing 57–59% of MATH-500 reasoning tokens while raising accuracy by 9–16 percentage points. Crucially, MMLU accuracy is fully preserved after training (73.2→73.3 for 8B, 76.9→76.9 for 14B), confirming that on-policy self-distillation does not degrade general capabilities.

5.3 Compression Naturally Adapts to Problem Difficulty

A key design question for any compression method is how to allocate budget across problems of varying difficulty. Prior RL methods estimate difficulty from rollout pass rates (Wan et al., 2026; Chen et al., 2025) or train separate difficulty classifiers. OPSDC requires none of this: **difficulty adaptation emerges for free from the KL objective.**

Table 2 quantifies this effect using benchmarks as a difficulty proxy.

Finding 2: Compression naturally adapts to difficulty. On the easiest benchmark (MATH-500), OPSDC compresses by **56.5–58.8%**. On the hardest (AIME 2025), compression drops to **35.2–35.7%**, a $\sim 1.6\times$ **ratio** that emerges automatically from the KL objective (see §3.3).

Larger models gain more from distillation. A curious pattern emerges across model scales. Qwen3-14B starts from *lower* base accuracy on MATH-500 (70.0% vs. 77.7% for 8B) yet achieves comparable post-distillation accuracy (86.1% vs. 86.6%), a +16.1 point gain versus +8.9 for 8B. More strikingly, on AIME 2024, the 14B model *improves* by 10.4 points (65.8→76.3) while the 8B model slightly declines (72.5→69.6). Why? Larger models follow instructions better, making the concise teacher a stronger signal, and they have more redundancy to shed.

5.4 Self-Distillation Does Not Collapse Model Entropy

Finding 3: Self-distillation preserves what RL destroys. A central concern with reasoning compression is *entropy collapse*: RL methods with length penalties systematically suppress high-entropy “exploratory” tokens (“Wait,” “Alternatively,” “Let me reconsider. . .”) that are critical for solving hard problems (Wang et al., 2025b). Figure 3 shows that OPSDC avoids this entirely—entropy remains stable throughout training. The model learns to *choose* conciseness rather than being forced into it.

This stability follows from the mode-seeking property of reverse KL (§3.2): the student is penalized for placing mass where the teacher assigns low probability, but *not* for maintaining mass where the teacher is also uncertain. High-entropy reasoning tokens that the concise teacher retains are therefore preserved. In contrast, RL length penalties reward shorter outputs regardless of token informativeness, collapsing entropy indiscriminately.

³All benchmarks are evaluated using the math answer grading utility from veRL (Sheng et al., 2025): https://github.com/verl-project/verl/blob/main/verl/utils/reward_score/math_dapo.py.

⁴MMLU is evaluated using the Language Model Evaluation Harness (Gao et al., 2021): <https://github.com/EleutherAI/lm-evaluation-harness>.

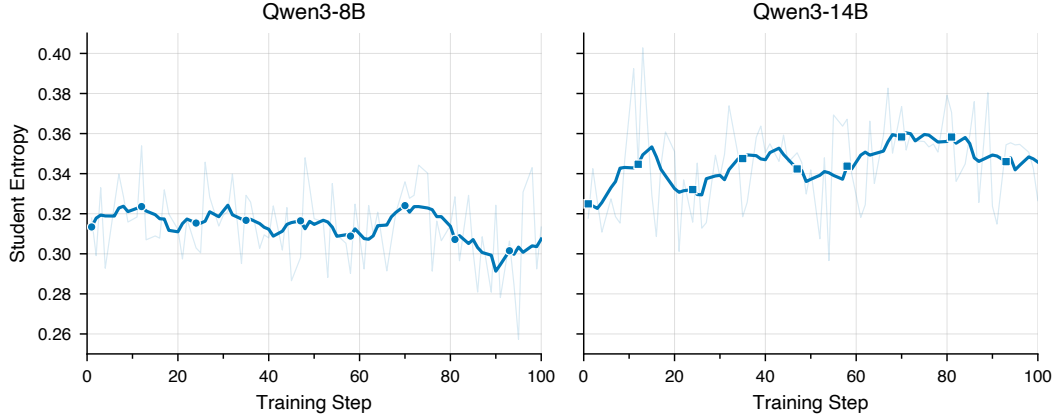


Figure 3: **Self-distillation preserves model entropy throughout training.** Average per-token entropy of the student model over training steps for Qwen3-8B (left) and Qwen3-14B (right) using the concise instruction. Unlike RL with length penalties, which drives entropy toward collapse (Liu et al., 2025; Cui et al., 2025), OPSDC maintains stable entropy: the model learns to be concise without losing its exploratory capacity.

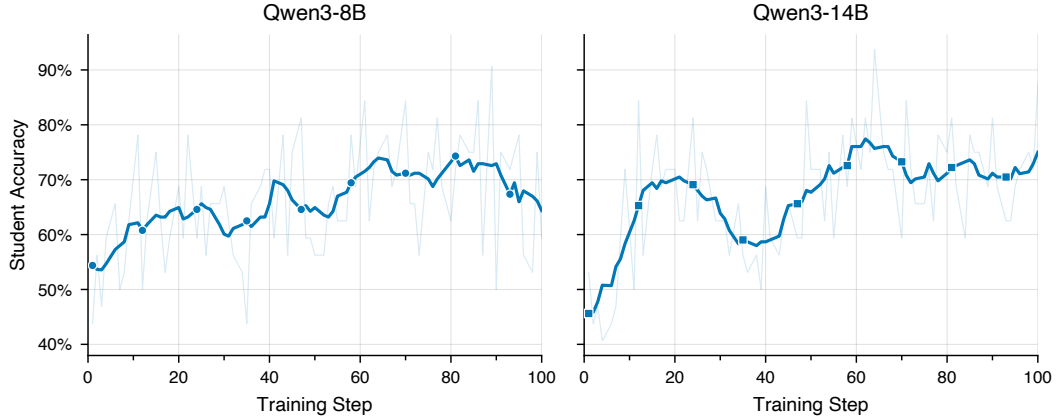


Figure 4: **Student mean accuracy on training data increases during self-distillation.** Qwen3-8B improves from $\sim 52\%$ to $\sim 66\%$ and Qwen3-14B from $\sim 46\%$ to $\sim 72\%$, despite no correctness reward. The concise teacher’s implicit reward reshapes the student’s output distribution, concentrating probability mass on direct, correct reasoning paths.

5.5 Why Does Compression Improve Accuracy?

Finding 4: Compression improves accuracy via implicit reward shaping. The reverse KL objective implicitly rewards concise, correct tokens (Theorem 1) and concentrates probability mass on the teacher’s preferred mode. Training-time accuracy increases monotonically (Figure 4)—from $\sim 52\%$ to $\sim 66\%$ for Qwen3-8B and $\sim 46\%$ to $\sim 72\%$ for Qwen3-14B—despite receiving no correctness reward.

The mode-seeking property of reverse KL (§3.2) drives the student toward the teacher’s preferred mode (Gu et al., 2023; Li et al., 2025a), concentrating probability mass on direct, correct reasoning paths. Since each additional token is a potential point of failure (Chen et al., 2024)—an effect we formalize in Proposition 3—compression simultaneously shortens traces and reduces error accumulation. Figure 12 in Appendix H provides side-by-side comparisons.

5.6 Qualitative Examples: Wrong-to-Correct Transitions

The following two examples illustrate how OPSDC simultaneously compresses reasoning and corrects errors. In each case, the base model Qwen3-8B produces a verbose, incorrect response, while the OPSDC-trained model produces a concise, correct one (more examples in Appendix H)

Problem 1: *Suzanne walks four miles every third day. What is the fewest number of miles she can walk in February?* (Correct answer: 36)

Qwen3-8B model (3,042 tokens)	OPSDC (1,220 tokens, 60% reduction)
<pre><think> ...I need to find the fewest number of miles she can walk in February... February has 28 days. She walks every third day. If she starts on day 28, she walks once. If she starts on day 27, she walks once. If she starts on day 26, she walks once... So the minimum number of walks is 1, leading to 4 miles. But wait, is that correct? Let me think again... If she starts on day 25, she walks on day 25 and 28. That's two walks. Therefore, starting on day 26, 27, or 28, she walks only once... But the problem says "every third day". Does that mean...? [500+ tokens debating interpretation]... I think the answer is 4. </think> Answer: 4 ✗ Misinterprets "fewest miles" as choosing an optimal start day for 1 walk, ignoring that "every third day" constrains the walking schedule.</pre>	<pre><think> Suzanne walks four miles every third day. February has 28 or 29 days; to minimize miles, consider 28 days. She walks every third day. Starting on day 1: days 1,4,7,...,28 ⇒ 10 walks. Starting on day 2: days 2,5,...,26 ⇒ 9 walks. Starting on day 3: days 3,6,...,27 ⇒ 9 walks. Minimum is 9 walks regardless of starting day (also holds for 29-day February). 9 × 4 = 36 miles. </think> Answer: 36 ✓</pre>

Problem 2: *A polynomial with integer coefficients is of the form $2x^4 + a_3x^3 + a_2x^2 + a_1x + 1 = 0$. Find the number of different possible rational roots.* (Correct answer: 4)

Qwen3-8B model (3,520 tokens)	OPSDC (390 tokens, 89% reduction)
<pre><think> ...By the Rational Root Theorem, possible roots are ±1, ±1/2... Wait, but does that mean these four are all the possibilities? Let me check... Can 1 be a root? Let's plug in...[verifies with concrete coefficients]... Yes. Can -1 be a root? Let's plug in...[verifies]... Yes. Can 1/2 be a root? Let me compute each term...[30+ lines of algebra]... Yes. Can -1/2 be a root? [30+ lines of algebra]... Yes. Therefore the answer is 4. But let me think again...[re-derives from scratch]... Yes, 4. </think> [Full formatted solution repeating all steps from <think> block: Rational Root Theorem statement, factor enumeration, verification of all four candidates...] $\text{Answer: } 4$ Correct reasoning but answer extraction fails due to verbose post-</think> formatting.</pre>	<pre><think> By the Rational Root Theorem, any rational root p/q must have $p \mid 1$ and $q \mid 2$. Factors of 1: ± 1. Factors of 2: $\pm 1, \pm 2$. Possible roots: $\pm 1, \pm 1/2$. That gives four possibilities. Double-check: constant term is 1, leading coefficient is 2. The fractions reduce to $\pm 1, \pm 1/2$. Four distinct values. </think> Answer: 4 ✓</pre>

Figure 5: Problem 1 illustrates how excessive deliberation leads to a genuine reasoning error: the base model talks itself into a wrong interpretation. Problem 2 shows a format failure: correct reasoning is buried in 3,500 tokens of redundant verification and post- </think> repetition, causing answer extraction to fail. In both cases, compression eliminates the noise that caused the error.

5.7 Ablation Study

5.7.1 Do Quantitative Reduction Targets Outperform Qualitative Instructions?

Our default teacher simply says “be concise.” But what if we gave it a number? Telling the model to “use 50% fewer tokens” seems more precise—surely that would compress harder? We investigate

Soft Budget Teacher Prompt $\pi_{\bar{\theta}}(\cdot \mid x, c_p)$
Soft budget instruction c_p: Solve the following math problem correctly using $p\%$ fewer tokens than you normally would. Be more concise -- cut unnecessary elaboration, redundant steps, and verbose explanations while preserving correctness. The last line of your response should be of the form Answer: \$Answer (without quotes) where \$Answer is the answer to the problem. Find all real numbers x such that $x^3 - 6x^2 + 11x - 6 = 0$. Remember to put your answer on its own line after "Answer:".

Figure 6: **Soft budget teacher prompt.** Unlike the qualitative conciseness instruction in Figure 2, the soft budget variant specifies a quantitative reduction target $p \in \{20, 50, 80\}$. The student prompt remains unchanged (Figure 2, top).

Table 3: **Qualitative conciseness instructions outperform quantitative reduction targets on accuracy (30K token budget).** All variants use periodic teacher update ($M=50$) at step 100. Soft budgets achieve higher compression but substantially lower accuracy than the concise instruction, particularly on competition-level benchmarks. Accuracy (Acc, %), token reduction (Red., %), and accuracy change vs. base model (Δ Acc, pp).

Context	MATH-500			AIME 2024			AIME 2025		
	Acc	Red.	Δ Acc	Acc	Red.	Δ Acc	Acc	Red.	Δ Acc
<i>Qwen3-8B</i>									
Concise	86.6	58.8%	+8.9	69.6	35.4%	−2.9	57.1	35.7%	− 5.4
Soft ($p=20\%$)	86.2	60.1%	+8.6	70.8	39.5%	− 1.7	52.9	33.7%	−9.6
Soft ($p=50\%$)	85.9	60.4%	+8.2	63.8	36.3%	−8.8	52.1	36.2%	−10.4
Soft ($p=80\%$)	84.1	63.8%	+6.5	67.9	41.5%	−4.6	49.6	39.5%	−12.9
<i>Qwen3-14B</i>									
Concise	86.1	56.5%	+16.1	76.3	41.0%	+10.5	61.7	35.2%	− 5.4
Soft ($p=20\%$)	80.7	67.8%	+10.8	67.1	47.8%	+1.3	57.9	45.8%	−9.2
Soft ($p=50\%$)	80.7	67.2%	+10.8	67.1	49.7%	+1.3	57.5	48.8%	−9.6
Soft ($p=80\%$)	79.8	68.9%	+9.9	68.3	50.8%	+2.5	54.6	50.7%	−12.5

this by comparing four context variants, all trained with the same periodic teacher update ($M=50$): the static conciseness instruction and soft budget targets at $p \in \{20\%, 50\%, 80\%\}$. The soft budget teacher prompt (Figure 6) replaces the qualitative conciseness instruction with a specific reduction target while keeping all other aspects identical.

Table 3 reveals a clear **compression–accuracy tradeoff across context variants**: soft budgets achieve higher compression but substantially lower accuracy than the qualitative concise instruction.

Soft budgets compress more aggressively but sacrifice accuracy. On MATH-500, $p=80\%$ achieves **63.8%** token reduction for Qwen3-8B (versus the concise instruction’s 58.8%), but accuracy drops from 86.6% to 84.1%. The gap widens dramatically on competition-level benchmarks: for Qwen3-14B on AIME 2024, the concise instruction achieves 76.3% accuracy while all soft budget variants cluster around 67–68%. The concise instruction achieves the best accuracy on 5 of 6 model–benchmark combinations. The sole exception is Qwen3-8B on AIME 2024, where $p=20\%$ reaches 70.8% versus 69.6% for the concise instruction, a difference within evaluation noise.

Compression monotonically increases with p , but accuracy does not. For Qwen3-14B, token reduction on AIME 2024 increases with the target: $p=20\%$ achieves 47.8%, $p=50\%$ achieves 49.7%, and $p=80\%$ achieves 50.8%. However, the best soft-budget accuracy on AIME 2024 comes from $p=80\%$ (68.3%), not $p=20\%$ (67.1%), suggesting that the relationship between reduction target and accuracy is non-monotonic.

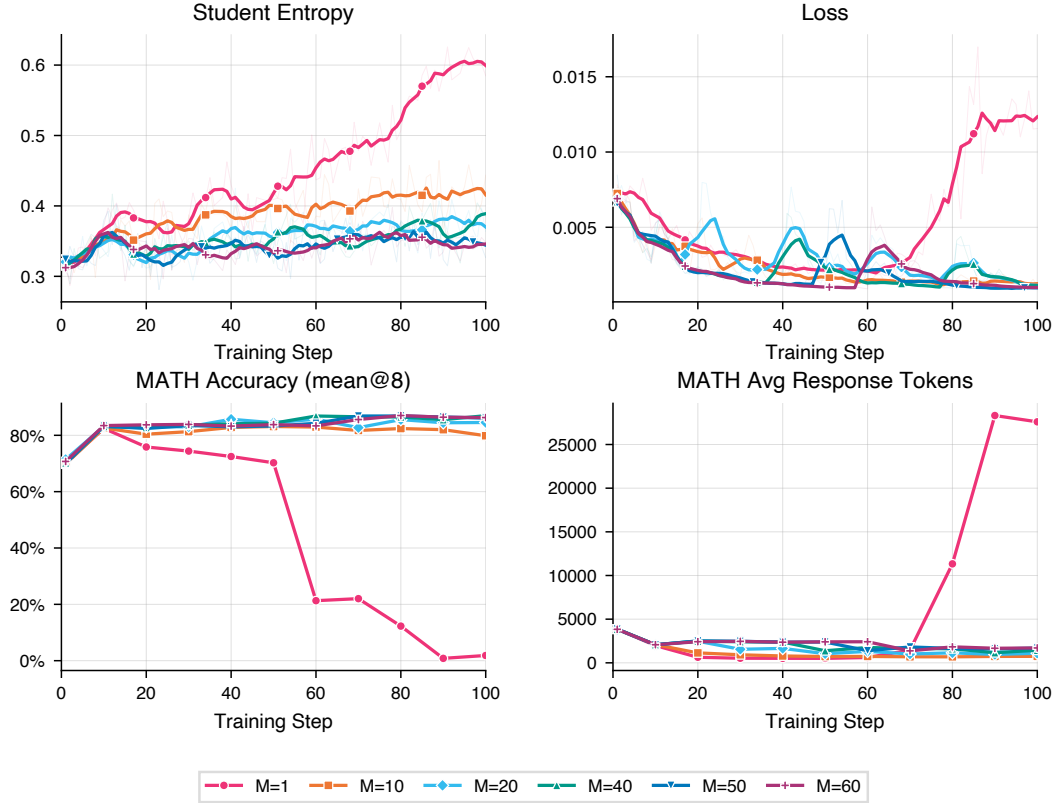


Figure 7: **Teacher update interval M controls the stability–compression trade-off.** Accuracy (left) and output entropy (right) over 100 training steps for Qwen3-14B on MATH-500 with varying M . $M=1$ (updating every step) causes entropy explosion and accuracy collapse to $\sim 2\%$ by step 100, consistent with the instability observed by Shenfeld et al. (2026). $M \in \{40, 50, 60\}$ produce stable trajectories reaching $\sim 86\text{--}87\%$ accuracy. $M=10$ peaks early then degrades, while $M=20$ remains competitive but shows mild entropy drift. All experiments use the qualitative concise instruction.

These results recommend the **qualitative concise instruction as the default**: it achieves the best accuracy, compresses substantially (57–59% on MATH-500), and—crucially—remains stable under extended training. The lesson: vague instructions make better teachers than precise ones.

5.7.2 How Sensitive Is Compression to the Teacher Update Interval?

The teacher update interval M (Eq. 2) controls how frequently the teacher weights are synchronized with the student. A larger M provides a more stable distillation target but limits progressive compression; a smaller M pushes compression further but risks training instability when the teacher changes too rapidly. We sweep $M \in \{1, 10, 20, 40, 50, 60\}$ using Qwen3-14B with the qualitative concise instruction on MATH-500.

Figure 7 reveals three distinct regimes:

$M=1$ is catastrophically unstable. Updating the teacher after every gradient step causes entropy to explode from ~ 0.32 to ~ 0.58 and accuracy to collapse from a peak of $\sim 82\%$ (step 10) to $\sim 2\%$ (step 100). This mirrors the finding of Shenfeld et al. (2026) that overly aggressive teacher updates create a moving target problem: the student chases a teacher that is itself changing in response to the student’s updates, leading to a positive feedback loop of increasingly degenerate outputs.

$M \in \{40, 50, 60\}$ form a stable plateau. These intervals produce similar accuracy trajectories, all reaching $\sim 86\text{--}87\%$ by step 100 with entropy remaining stable around $\sim 0.33\text{--}0.39$. The method is robust to the exact choice of M within this range: the teacher remains stable long enough for the student to meaningfully converge toward the current target before the next refresh.

$M=10$ **degrades**; $M=20$ is **borderline**. $M=10$ peaks at $\sim 83\%$ accuracy around step 50 but declines to $\sim 80\%$ by step 100, with entropy drifting up to ~ 0.44 . $M=20$ performs better (84.5% at step 100) but still trails the $M \geq 40$ regime by 2–3 percentage points.

Based on these results, we use $M=50$ for all other experiments in this paper, as it sits comfortably in the stable plateau while allowing progressive compression through periodic teacher refresh.

6 Limitations and Future Work

We discuss the scope of the current study and natural directions for future work.

Instruction-following as an enabler. OPSDC leverages the base model’s ability to follow conciseness instructions. Larger models with stronger instruction-following capabilities benefit more: Qwen3-14B achieves a +16.1 point accuracy gain versus +8.9 for Qwen3-8B on MATH-500. This positive correlation with model scale suggests that OPSDC will become *more* effective as foundation models continue to improve. Investigating the minimum capability threshold for effective self-distillation is an interesting direction for future work.

Progressive compression dynamics. Our periodic teacher update (Eq. 2) successfully pushes compression beyond a frozen teacher, and we show that the method is robust across a wide range of update intervals ($M \in \{40, 50, 60\}$; Section 5.7.2). Naturally, the compression signal on harder benchmarks (e.g., AIME) is weaker because the teacher itself requires more extensive reasoning—a property we view as a feature of difficulty-adaptive compression (Section 3.3) rather than a limitation.

Scope of evaluation. This paper focuses on mathematical reasoning as a controlled testbed where accuracy can be verified precisely. OPSDC’s design is domain-agnostic—it requires only problem prompts and a conciseness instruction—making it directly applicable to other reasoning domains (e.g., code generation, scientific QA) where ground-truth verification is unavailable. We note that the key advantages of OPSDC (no ground-truth requirement, difficulty adaptivity, and entropy preservation) are *structural* properties that hold regardless of the evaluation domain. Extending the empirical evaluation to broader reasoning tasks is a natural next step.

Teacher quality characterization. Our experiments consistently show that the conciseness-conditioned teacher improves accuracy (Table 2), and the theoretical analysis (Theorem 2) provides formal bounds on accuracy preservation. A finer-grained characterization of when and why conciseness instructions improve versus degrade accuracy across different model families would further strengthen the understanding of self-distillation dynamics.

7 Conclusion

We set out to make reasoning models more concise. We ended up making them more accurate.

OPSDC shows that much of what reasoning models produce is not deliberation but *noise*—and noise compounds. Every unnecessary token is a chance to wander off course, to second-guess a correct answer, to introduce an error that propagates forward. By teaching models to skip the noise, we do not sacrifice depth; we *recover* it.

Two takeaways stand out. First, verbosity is not caution—it can be a source of compounding error. Second, models already possess a latent ability to be concise; on-policy self-distillation can make this behavior the default without sacrificing entropy or general capabilities.

Finally, OPSDC’s supervision is purely behavioral: a conciseness instruction and the model’s own rollouts. This suggests a path to compressing reasoning in domains where ground-truth answers or reliable verifiers are unavailable, as long as the model can follow the desired instruction.

References

P. Aggarwal and S. Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.

- Y. Xu, H. Sang, Z. Zhou, R. He, and Z. Wang. Overconfident errors need stronger correction: Asymmetric confidence penalties for reinforcement learning. *arXiv preprint arXiv:2602.21420*, 2026.
- B. Hou, Y. Zhang, J. Ji, Y. Liu, K. Qian, J. Andreas, and S. Chang. Thinkprune: Pruning long chain-of-thought of llms via reinforcement learning. *arXiv preprint arXiv:2504.01296*, 2025.
- W. Chen, J. Yuan, T. Jin, N. Ding, H. Chen, Z. Liu, and M. Sun. The overthinker’s diet: Cutting token calories with difficulty-aware training. *arXiv preprint arXiv:2505.19217*, 2025.
- W.-R. Chen, V. Kothapalli, A. Fatahibaarzi, H. Sang, S. Tang, Q. Song, Z. Wang, and M. Abdul-Mageed. Distilling the essence: Efficient reasoning distillation via sequence truncation. *arXiv preprint arXiv:2512.21002*, 2025b.
- X. Chen, J. Xu, T. Liang, Z. He, J. Pang, D. Yu, L. Song, Q. Liu, M. Zhou, Z. Zhang, et al. Do not think that much for $2+3=?$ on the overthinking of o1-like llms. *arXiv preprint arXiv:2412.21187*, 2024.
- G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- G. Cui, Y. Zhang, J. Chen, L. Yuan, Z. Wang, Y. Zuo, H. Li, Y. Fan, H. Chen, W. Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.
- Y. Du, S. Zhao, Y. Gao, D. Zhao, Q. Lin, M. Ma, J. Li, Y. Jiang, K. He, Q. Xu, et al. S3cot: Self-sampled succinct reasoning enables efficient chain-of-thought llms. *arXiv preprint arXiv:2602.01982*, 2026.
- L. Gao, J. Tow, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, K. McDonell, N. Muennighoff, et al. A framework for few-shot language model evaluation, 2021.
- Y. Gu, L. Dong, F. Wei, and M. Huang. Minillm: Knowledge distillation of large language models. In *arXiv preprint arXiv:2306.08543*, 2023.
- D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- K. Huang, S. Liu, X. Hu, T. Xu, L. Bao, and X. Xia. Reasoning efficiently through adaptive chain-of-thought compression: A self-optimizing framework. *arXiv preprint arXiv:2509.14093*, 2025.
- J. Hübotter, F. Lübeck, L. Behric, A. Baumann, M. Bagatella, D. Marta, I. Hakimi, I. Shenfeld, T. K. Buening, C. Guestrin, et al. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.
- A. Jaech, A. Kalai, A. Lerer, A. Richardson, A. El-Kishky, A. Low, A. Helyar, A. Madry, A. Beutel, A. Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- L. Li, J. Hao, J. K. Liu, Z. Zhou, Y. Miao, W. Pang, X. Tan, W. Chu, Z. Wang, S. Pan, et al. The choice of divergence: A neglected key to mitigating diversity collapse in reinforcement learning with verifiable reward. *arXiv preprint arXiv:2509.07430*, 2025a.
- Y. Li, L. Ma, J. Zhang, L. Tang, W. Zhang, and G. Luo. Leash: Adaptive length penalty and reward shaping for efficient large reasoning model. *arXiv preprint arXiv:2512.21540*, 2025b.
- Y. Li, B. Bergner, Y. Zhao, V. P. Patil, B. Chen, and C. Wang. Steering large reasoning models towards concise reasoning via flow matching. *arXiv preprint arXiv:2602.05539*, 2026.
- W. Lin, X. Li, Z. Yang, X. Fu, H.-L. Zhen, Y. Wang, X. Yu, W. Liu, X. Li, and M. Yuan. Trimr: Verifier-based training-free thinking compression for efficient test-time scaling. *arXiv preprint arXiv:2505.17155*, 2025.

- S. Liu, X. Dong, X. Lu, S. Diao, M. Liu, M. Chen, H. Yin, Y. Wang, K. Cheng, Y. Choi, et al. DLER: Doing length penalty right – incentivizing more intelligence per token via reinforcement learning. *arXiv preprint arXiv:2510.15110*, 2025.
- N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. B. Hashimoto. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20286–20332, 2025.
- I. Shenfeld, M. Damani, J. Hübotter, and P. Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026.
- G. Sheng, C. Zhang, Z. Ye, X. Wu, W. Zhang, R. Zhang, Y. Peng, H. Lin, and C. Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.
- C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- Q. Wan, Z. Xu, L. Wei, X. Shen, and J. Sun. Mitigating overthinking in large reasoning models via difficulty-aware reinforcement learning. *arXiv preprint arXiv:2601.21418*, 2026.
- C. Wang, Y. Feng, D. Chen, Z. Chu, R. Krishna, and T. Zhou. Wait, we don’t need to" wait"! removing thinking tokens improves reasoning efficiency. *arXiv preprint arXiv:2506.08343*, 2025a.
- S. Wang, L. Yu, C. Gao, C. Zheng, S. Liu, R. Lu, K. Dang, X. Chen, J. Yang, Z. Zhang, et al. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning. *arXiv preprint arXiv:2506.01939*, 2025b.
- Y. Wu, J. Shi, B. Wu, J. Zhang, X. Lin, N. Tang, and Y. Luo. Concise reasoning, big gains: Pruning long reasoning trace with difficulty-aware prompting. *arXiv preprint arXiv:2505.19716*, 2025.
- H. Xia, C. T. Leong, W. Wang, Y. Li, and W. Li. Tokenskip: Controllable chain-of-thought compression in llms. *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 3351–3363, 2025.
- S. Xu, W. Xie, L. Zhao, and P. He. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*, 2025.
- A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- T. Ye, L. Dong, X. Wu, S. Huang, and F. Wei. On-policy context distillation for language models. *arXiv preprint arXiv:2602.12275*, 2026.
- Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- S. Zhao, Z. Xie, M. Liu, J. Huang, G. Pang, F. Chen, and A. Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.
- L. Zheng, L. Yin, Z. Xie, C. L. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.

A Theoretical Analysis

We provide a formal analysis of OPSDC’s key properties: the connection between the per-token training loss and sequence-level divergence (Section A.1), accuracy preservation guarantees (Section A.2), formalization of difficulty-adaptive compression (Section A.3), forgetting bounds relative to the base model (Section A.4), and a probabilistic model of how compression reduces compounding errors (Section A.5).

A.1 Sequence-Level Divergence and the Training Objective

We first establish that the per-token OPSDC objective (Eq. 1) is equivalent to minimizing the sequence-level KL divergence between student and teacher. This identification enables the application of standard information-theoretic tools to the per-token loss.

Lemma 1 (Chain rule of KL for autoregressive models). *For autoregressive distributions $q(y \mid x) = \prod_{t=1}^{|y|} q(y_t \mid x, y_{<t})$ and $p(y \mid x) = \prod_{t=1}^{|y|} p(y_t \mid x, y_{<t})$ over the same token vocabulary, the sequence-level KL divergence decomposes as:*

$$D_{\text{KL}}(q(\cdot \mid x) \parallel p(\cdot \mid x)) = \mathbb{E}_{y \sim q} \left[\sum_{t=1}^{|y|} D_{\text{KL}}(q(\cdot \mid x, y_{<t}) \parallel p(\cdot \mid x, y_{<t})) \right]. \quad (8)$$

Proof. By definition of KL divergence and the autoregressive factorization:

$$D_{\text{KL}}(q \parallel p) = \mathbb{E}_{y \sim q} \left[\log \frac{q(y \mid x)}{p(y \mid x)} \right] = \mathbb{E}_{y \sim q} \left[\sum_{t=1}^{|y|} \log \frac{q(y_t \mid x, y_{<t})}{p(y_t \mid x, y_{<t})} \right] \quad (9)$$

$$= \mathbb{E}_{y \sim q} \left[\sum_{t=1}^{|y|} D_{\text{KL}}(q(\cdot \mid x, y_{<t}) \parallel p(\cdot \mid x, y_{<t})) \right], \quad (10)$$

where the second equality uses $\log \prod_t = \sum_t \log$, and the last step recognizes each summand as a per-token KL divergence evaluated at the sampled prefix $y_{<t}$. $\square$

Corollary 1. *Identifying $q = \pi_\theta(\cdot \mid x)$ and $p = \pi_{\bar{\theta}}(\cdot \mid x, c)$, the OPSDC training loss (Eq. 1) equals the expected sequence-level KL divergence:*

$$\mathcal{L}(\theta) = \mathbb{E}_{x \sim \mathcal{D}} [D_{\text{KL}}(\pi_\theta(\cdot \mid x) \parallel \pi_{\bar{\theta}}(\cdot \mid x, c))]. \quad (11)$$

A.2 Accuracy Preservation under Compression

We show that if self-distillation converges (the training loss is small) and the concise teacher preserves accuracy, then the student’s accuracy is guaranteed to remain close to the base model’s.

Definition 1 (Accuracy). For a problem distribution $\mathcal{D}$ with correct-answer sets $\{A(x)\}_{x \in \mathcal{D}}$, the accuracy of policy π is $\text{Acc}(\pi) = \mathbb{E}_{x \sim \mathcal{D}} [\pi(A(x) \mid x)]$, where $\pi(A(x) \mid x) = \sum_{y \in A(x)} \pi(y \mid x)$.

Theorem 2 (Accuracy preservation). *Let π_{θ^*} denote the converged student with training loss $\mathcal{L}(\theta^*) \leq \epsilon_{\text{KL}}$. Suppose the concise teacher preserves accuracy relative to the base model:*

$$\text{Acc}(\pi_{\bar{\theta}}(\cdot \mid \cdot, c)) \geq \text{Acc}(\pi_{\bar{\theta}}) - \epsilon_T. \quad (12)$$

Then the student satisfies:

$$\text{Acc}(\pi_{\theta^*}) \geq \text{Acc}(\pi_{\bar{\theta}}) - \epsilon_T - \sqrt{\frac{\epsilon_{\text{KL}}}{2}}. \quad (13)$$

Proof. By Corollary 1, we have $\mathbb{E}_{x \sim \mathcal{D}} [D_{\text{KL}}(\pi_{\theta^*}(\cdot \mid x) \parallel \pi_{\bar{\theta}}(\cdot \mid x, c))] \leq \epsilon_{\text{KL}}$.

Step 1: KL to total variation. For each problem x , Pinsker’s inequality gives:

$$d_{\text{TV}}(\pi_{\theta^*}(\cdot \mid x), \pi_{\bar{\theta}}(\cdot \mid x, c)) \leq \sqrt{\frac{1}{2} D_{\text{KL}}(\pi_{\theta^*}(\cdot \mid x) \parallel \pi_{\bar{\theta}}(\cdot \mid x, c))}. \quad (14)$$

Taking expectations over $x \sim \mathcal{D}$ and applying Jensen's inequality (using concavity of $\sqrt{\cdot}$):

$$\mathbb{E}_x[d_{\text{TV}}(\pi_{\theta^*}(\cdot | x), \pi_{\bar{\theta}}(\cdot | x, c))] \leq \sqrt{\frac{1}{2} \mathbb{E}_x[D_{\text{KL}}(\pi_{\theta^*}(\cdot | x) \| \pi_{\bar{\theta}}(\cdot | x, c))]} \leq \sqrt{\frac{\epsilon_{\text{KL}}}{2}}. \quad (15)$$

Step 2: Total variation to accuracy. Since total variation bounds the difference in probability of any event, in particular the correctness event $A(x)$:

$$|\pi_{\theta^*}(A(x) | x) - \pi_{\bar{\theta}}(A(x) | x, c)| \leq d_{\text{TV}}(\pi_{\theta^*}(\cdot | x), \pi_{\bar{\theta}}(\cdot | x, c)). \quad (16)$$

Taking expectations over $x \sim \mathcal{D}$ and using $|\mathbb{E}[f]| \leq \mathbb{E}[|f|]$:

$$|\text{Acc}(\pi_{\theta^*}) - \text{Acc}(\pi_{\bar{\theta}}(\cdot | \cdot, c))| \leq \mathbb{E}_x[d_{\text{TV}}(\pi_{\theta^*}(\cdot | x), \pi_{\bar{\theta}}(\cdot | x, c))] \leq \sqrt{\frac{\epsilon_{\text{KL}}}{2}}. \quad (17)$$

Step 3: Combine with teacher quality. From the assumption $\text{Acc}(\pi_{\bar{\theta}}(\cdot | \cdot, c)) \geq \text{Acc}(\pi_{\bar{\theta}}) - \epsilon_T$:

$$\text{Acc}(\pi_{\theta^*}) \geq \text{Acc}(\pi_{\bar{\theta}}(\cdot | \cdot, c)) - \sqrt{\frac{\epsilon_{\text{KL}}}{2}} \geq \text{Acc}(\pi_{\bar{\theta}}) - \epsilon_T - \sqrt{\frac{\epsilon_{\text{KL}}}{2}}. \quad \square$$

Remark 2 (When the bound is vacuous, and why that is informative). *Theorem 2 identifies two independent sources of potential accuracy loss: the teacher accuracy gap ϵ_T and the student-teacher divergence $\sqrt{\epsilon_{\text{KL}}/2}$. In our experiments, ϵ_T is consistently negative (the concise teacher is more accurate than the base model), making the bound vacuous in the traditional sense. This is informative rather than a weakness: it reveals why OPSDC improves accuracy. The bound becomes $\text{Acc}(\pi_{\theta^*}) \geq \text{Acc}(\pi_{\bar{\theta}}) + |\epsilon_T| - \sqrt{\epsilon_{\text{KL}}/2}$, so accuracy improves whenever the teacher's accuracy gain exceeds the distillation gap. The theorem is most useful in the regime of aggressive compression where ϵ_T might turn positive; it then quantifies the worst-case accuracy degradation.*

A.3 Difficulty-Adaptive Compression

We formalize the empirical observation (Table 2) that OPSDC compresses easy problems aggressively while preserving reasoning on hard problems.

Definition 2 (Essential and compressible tokens). For problem x and student rollout $y \sim \pi_{\theta}(\cdot | x)$, classify each token position t based on the implicit reward sign (Theorem 1):

$$\mathcal{E}(x, y) = \{t : \pi_{\bar{\theta}}(y_t | x, c, y_{<t}) \geq \pi_{\theta}(y_t | x, y_{<t})\} \quad (\text{essential: } r(y_t, x) \geq 0), \quad (18)$$

$$\mathcal{C}(x, y) = \{t : \pi_{\bar{\theta}}(y_t | x, c, y_{<t}) < \pi_{\theta}(y_t | x, y_{<t})\} \quad (\text{compressible: } r(y_t, x) < 0). \quad (19)$$

Proposition 1 (Difficulty-adaptive compression signal). *Let $d(x) \in [0, 1]$ denote problem difficulty, defined as the base model's failure rate $d(x) = 1 - \pi_{\theta_0}(A(x) | x)$. Assume:*

- (A1) **Essential fraction increases with difficulty:** *The expected fraction of essential tokens $\rho(x) := \mathbb{E}_y[|\mathcal{E}(x, y)|/|y|]$ is non-decreasing in $d(x)$.*
- (A2) **Category-level KL is problem-independent:** *There exist constants $D_{\mathcal{E}}, D_{\mathcal{C}} > 0$ such that for all problems x :*

$$\mathbb{E}[D_{\text{KL}}(\pi_{\theta}(\cdot | x, y_{<t}) \| \pi_{\bar{\theta}}(\cdot | x, c, y_{<t})) | t \in \mathcal{C}(x, y)] = D_{\mathcal{C}}, \quad (20)$$

$$\mathbb{E}[D_{\text{KL}}(\pi_{\theta}(\cdot | x, y_{<t}) \| \pi_{\bar{\theta}}(\cdot | x, c, y_{<t})) | t \in \mathcal{E}(x, y)] = D_{\mathcal{E}}. \quad (21)$$

- (A3) **Compressible tokens carry strictly larger KL:** $D_{\mathcal{C}} > D_{\mathcal{E}}$.

Then the expected normalized compression signal

$$S(x) = \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} D_{\text{KL}}(\pi_{\theta}(\cdot | x, y_{<t}) \| \pi_{\bar{\theta}}(\cdot | x, c, y_{<t})) \right] \quad (22)$$

is non-increasing in $d(x)$.

Proof. Decompose the normalized KL into essential and compressible contributions. For a given rollout y :

$$\frac{1}{|y|} \sum_t D_{\text{KL}}(q_t \| p_t) = \underbrace{\frac{|\mathcal{E}|}{|y|} \cdot \bar{D}_{\mathcal{E}}}_{\text{essential term}} + \underbrace{\frac{|\mathcal{C}|}{|y|} \cdot \bar{D}_{\mathcal{C}}}_{\text{compressible term}}, \quad (23)$$

where $q_t = \pi_{\theta}(\cdot \mid x, y_{<t})$, $p_t = \pi_{\bar{\theta}}(\cdot \mid x, c, y_{<t})$, and $\bar{D}_{\mathcal{E}}, \bar{D}_{\mathcal{C}}$ denote the average per-token KL on essential and compressible tokens in this rollout, respectively. Taking expectations over y and applying assumption (A2):

$$S(x) = \rho(x) \cdot D_{\mathcal{E}} + (1 - \rho(x)) \cdot D_{\mathcal{C}} \quad (24)$$

$$= D_{\mathcal{C}} - \rho(x) \cdot (D_{\mathcal{C}} - D_{\mathcal{E}}). \quad (25)$$

By (A3), $D_{\mathcal{C}} - D_{\mathcal{E}} > 0$, so $S(x)$ is a strictly decreasing affine function of $\rho(x)$. Since $\rho(x)$ is non-decreasing in $d(x)$ by (A1), $S(x)$ is non-increasing in $d(x)$.

Quantitatively, for two problems with difficulties $d_1 < d_2$ (hence $\rho(x_1) \leq \rho(x_2)$ by A1):

$$S(x_1) - S(x_2) = (\rho(x_2) - \rho(x_1)) \cdot (D_{\mathcal{C}} - D_{\mathcal{E}}) \geq 0. \quad (26)$$

□

Remark 3. Assumption (A1), that harder problems have a larger fraction of essential reasoning steps, is the core structural assumption. It is empirically supported by the monotonic compression–difficulty relationship in Table 2: MATH-500 (57–59% compression, base accuracy 70–78%) versus AIME 2025 (~35% compression, base accuracy ~63–67%). Assumption (A2) posits that the average KL divergence on essential and compressible tokens depends on token category (essential vs. compressible) rather than on the specific problem. This is a modeling simplification; in practice, per-token KL values vary across problems, and the proposition should be interpreted as holding for the category-averaged quantities. Assumption (A3) states that compressible tokens exhibit a larger vocabulary-level KL divergence than essential tokens. Note that this does not follow directly from the per-token reward sign: the essential/compressible classification (Definition 2) is based on the probability of the sampled token y_t , whereas the KL divergence $D_{\text{KL}}(q_t \| p_t)$ is an expectation over the entire vocabulary at position t . Rather, (A3) is a structural modeling assumption, motivated by the intuition that at positions where the teacher concentrates mass differently from the student (compressible positions), the full distributional divergence tends to be larger than at positions where both distributions agree on the dominant token (essential positions).

A.4 Bounded Forgetting from the Base Model

A central advantage of on-policy self-distillation over off-policy SFT is controlled divergence from the original model. We formalize this through the *conciseness gap*.

Definition 3 (Conciseness gap). The conciseness gap of the base model π_{θ_0} under instruction c on input x is

$$\gamma(x) = d_{\text{TV}}(\pi_{\theta_0}(\cdot \mid x), \pi_{\theta_0}(\cdot \mid x, c)). \quad (27)$$

Proposition 2 (Bounded forgetting under on-policy self-distillation). Consider the first teacher window where $\bar{\theta} = \theta_0$ (frozen teacher). If the converged OPSDC loss satisfies $\mathcal{L}(\theta^*) \leq \epsilon_{\text{KL}}$, then:

$$\mathbb{E}_{x \sim \mathcal{D}}[d_{\text{TV}}(\pi_{\theta^*}(\cdot \mid x), \pi_{\theta_0}(\cdot \mid x))] \leq \sqrt{\frac{\epsilon_{\text{KL}}}{2}} + \mathbb{E}_{x \sim \mathcal{D}}[\gamma(x)]. \quad (28)$$

For subsequent windows with periodic teacher update, the same bound holds with θ_0 replaced by the teacher weights $\bar{\theta}$ at the start of that window. Moreover, $\gamma(x)$ is difficulty-adaptive: for hard problems where the conciseness instruction has little effect, $\gamma(x) \approx 0$, so forgetting is minimal.

Proof. By the triangle inequality for total variation distance:

$$d_{\text{TV}}(\pi_{\theta^*}(\cdot \mid x), \pi_{\theta_0}(\cdot \mid x)) \leq \underbrace{d_{\text{TV}}(\pi_{\theta^*}(\cdot \mid x), \pi_{\theta_0}(\cdot \mid x, c))}_{\text{student-teacher gap}} + \underbrace{d_{\text{TV}}(\pi_{\theta_0}(\cdot \mid x, c), \pi_{\theta_0}(\cdot \mid x))}_{\gamma(x)}. \quad (29)$$

The student–teacher gap is bounded via Pinsker’s inequality. By Corollary 1, $\mathbb{E}_x[D_{\text{KL}}(\pi_{\theta^*}(\cdot | x) || \pi_{\theta_0}(\cdot | x, c))] \leq \epsilon_{\text{KL}}$. Applying Pinsker to each x and Jensen’s inequality over $x \sim \mathcal{D}$:

$$\mathbb{E}_x[d_{\text{TV}}(\pi_{\theta^*}(\cdot | x), \pi_{\theta_0}(\cdot | x, c))] \leq \sqrt{\frac{\epsilon_{\text{KL}}}{2}}. \quad (30)$$

Taking expectations on both sides of the triangle inequality yields (28).

For the difficulty-adaptive claim: on hard problems, the conciseness instruction cannot substantially alter the output distribution because most reasoning steps are essential. Thus $\pi_{\theta_0}(\cdot | x, c) \approx \pi_{\theta_0}(\cdot | x)$, giving $\gamma(x) \approx 0$. $\square$

Remark 4 (Comparison with off-policy SFT). *Standard off-policy SFT minimizes $-\mathbb{E}_{(x,y) \sim \mathcal{D}_T}[\log \pi_{\theta}(y | x)]$ on a fixed teacher dataset $\mathcal{D}_T$. The analogous forgetting decomposition is:*

$$d_{\text{TV}}(\pi_{\theta_{\text{SFT}}}(\cdot | x), \pi_{\theta_0}(\cdot | x)) \leq d_{\text{TV}}(\pi_{\theta_{\text{SFT}}}(\cdot | x), \mathcal{D}_T(\cdot | x)) + d_{\text{TV}}(\mathcal{D}_T(\cdot | x), \pi_{\theta_0}(\cdot | x)). \quad (31)$$

The second term measures the distribution mismatch between the teacher’s data and the base model’s outputs, which can be substantially larger than the conciseness gap $\gamma(x)$, particularly when the teacher generates qualitatively different reasoning styles. Crucially, off-policy SFT drives $\pi_{\theta_{\text{SFT}}}$ toward $\mathcal{D}_T$ directly, while on-policy distillation generates data from the student’s own evolving distribution, inherently limiting divergence from the base model (Shenfeld et al., 2026).

A.5 Compression Reduces Compounding Error

We provide a simple probabilistic model that explains the most striking empirical finding: shorter reasoning traces can *improve* accuracy rather than degrade it.

Proposition 3 (Shorter traces reduce error accumulation). *Consider an autoregressive reasoning model where each token position independently introduces a reasoning error (an incorrect intermediate step that corrupts the final answer) with probability $p_{\text{err}} \in (0, 1)$. For a trace of length L , the probability of producing a correct answer is:*

$$\text{Acc}(L) = (1 - p_{\text{err}})^L. \quad (32)$$

If compression reduces the trace from L to αL tokens ($\alpha \in (0, 1)$) without increasing the per-token error rate, the accuracy ratio satisfies:

$$\frac{\text{Acc}(\alpha L)}{\text{Acc}(L)} = (1 - p_{\text{err}})^{-(1-\alpha)L} \geq 1 + (1 - \alpha)L \cdot p_{\text{err}}. \quad (33)$$

The accuracy improvement grows exponentially in the number of removed tokens $(1 - \alpha)L$.

Proof. Direct computation gives the exact ratio:

$$\frac{\text{Acc}(\alpha L)}{\text{Acc}(L)} = \frac{(1 - p_{\text{err}})^{\alpha L}}{(1 - p_{\text{err}})^L} = (1 - p_{\text{err}})^{-(1-\alpha)L}. \quad (34)$$

Let $m = (1 - \alpha)L > 0$. Since $\ln(1 - p) \leq -p$ for all $p \in (0, 1)$ (which follows from the concavity of $\ln$ and the tangent line at $p = 0$), we have $-\ln(1 - p_{\text{err}}) \geq p_{\text{err}}$, giving:

$$(1 - p_{\text{err}})^{-m} = e^{-m \ln(1 - p_{\text{err}})} \geq e^{m \cdot p_{\text{err}}} \geq 1 + m \cdot p_{\text{err}}, \quad (35)$$

where the final inequality is $e^u \geq 1 + u$ for all $u \geq 0$. $\square$

Remark 5 (Quantitative interpretation). *On MATH-500, the base model generates $L \approx 4,660$ tokens and OPSDC compresses to $\alpha \approx 0.41$, removing $(1 - \alpha)L \approx 2,750$ tokens. Even with a modest per-token error rate of $p_{\text{err}} = 10^{-4}$, the linear lower bound gives an accuracy ratio ≥ 1.28 , a 28% relative improvement. The exponential term $(1 - p_{\text{err}})^{-2750} \approx e^{0.28} \approx 1.32$ provides the tighter bound. On AIME benchmarks where $L \approx 14,000$ tokens, the same per-token error rate yields even larger potential gains per token removed.*

Remark 6 (Conservative nature of the independence assumption). *The independence assumption is conservative: in practice, reasoning errors compound because one incorrect intermediate step causes subsequent steps to build on a false premise, amplifying the error. Under positively correlated errors, the benefit of removing error-prone tokens exceeds the independent-error prediction, making Proposition 3 a lower bound on the improvement from compression. This explains why the empirical accuracy gains (e.g., 70.0 $\rightarrow$ 86.1 on MATH-500, 65.8 $\rightarrow$ 76.3 on AIME 2024 for Qwen3-14B) substantially exceed what the simple model predicts.*

B Survey of Reasoning Compression Methods

Table 4 summarizes 19 reasoning compression methods along four axes. This survey motivates the design of OPSDC by revealing the pervasive dependence on ground-truth answers and the rarity of difficulty-adaptive methods.

Table 4: Survey of 19 reasoning compression methods. LP = Length Penalty in reward; DD = Difficulty-Dependent; CA = Correct Answer required; HB = Hard Budget.

Method	LP	DD	CA	HB	Approach
L1	✓		✓	✓	RL
DiPO	✓	✓	✓		RL
TRAAC	✓	✓	✓		RL
DIET	✓	✓	✓		RL
DLER	✓	✓	✓		RL
Leash	✓		✓		RL
ORION	✓		✓		RL
AdaptThink		✓	✓		RL
SEER			✓		SFT
TokenSkip			✓	✓	SFT
V-Skip			✓		SFT
S3-CoT					Steering
DAP/LiteCoT		✓	✓		SFT
Extra-CoT			✓		SFT
CtrlCoT			✓	✓	SFT
Chain of Draft					Prompt
TrimR					Inference
NoWait					Inference
FlowSteer					Inference
OPSDC (Ours)		✓			Self-distill

C Extended Results Across Token Budgets

C.1 Results Under 8K Token Budget

Table 5 reports results under the efficient-serving token budget of 8,192 tokens. Because the base model frequently produces responses exceeding this limit on harder benchmarks (AIME 2024/2025), truncation disproportionately affects the verbose baseline, making the compression gains appear even larger. We include these results as a practical reference for deployment scenarios where strict token budgets are enforced.

Table 5: **Self-distillation results under the 8,192-token budget.** Same setup as Table 2 but with max response length capped at 8,192 tokens, representative of efficient serving constraints. Accuracy (Acc, mean over 8 samples, %), average reasoning token length (Len), and token reduction (Red., %).

Method	MATH-500			AIME 2024			AIME 2025		
	Acc	Len	Red.	Acc	Len	Red.	Acc	Len	Red.
<i>Qwen3-8B</i>									
Base Model	69.4	3,860	—	25.0	7,597	—	18.8	7,661	—
Concise prompt	78.1	2,514	34.9%	37.9	6,870	9.6%	29.2	7,140	6.8%
OPSDC	85.1	1,199	68.9%	54.6	5,114	32.7%	36.3	5,658	26.2%
<i>Qwen3-14B</i>									
Base Model	64.3	3,285	—	27.1	7,345	—	19.6	7,471	—
Concise prompt	81.8	2,059	37.3%	45.4	6,483	11.7%	30.8	6,852	8.3%
OPSDC	85.5	1,066	67.6%	57.5	4,599	37.4%	39.2	5,484	26.6%

C.2 Extended Training Under 30K Token Budget

Table 6 shows performance at training steps 100 and 200 under the 30K token budget, illustrating the accuracy–compression trade-off as training progresses beyond the sweet spot. At step 100 (our default checkpoint), both models achieve strong compression with accuracy improvements on MATH-500. Continuing to step 200 roughly doubles token reduction on AIME benchmarks (from $\sim 35\text{--}41\%$ to $\sim 51\text{--}53\%$) but at the cost of AIME accuracy, particularly for the harder AIME 2025. MATH-500 accuracy remains robust throughout, suggesting that compression on easier benchmarks is more sustainable.

Table 6: **Extended training results under the 30K token budget.** Performance at step 100 (default) and step 200, showing the accuracy–compression trade-off with continued training. Accuracy (Acc, mean over 8 samples, %), average reasoning token length (Len), and token reduction vs. base model (Red., %).

Method	MATH-500			AIME 2024			AIME 2025		
	Acc	Len	Red.	Acc	Len	Red.	Acc	Len	Red.
<i>Qwen3-8B</i>									
Base Model	77.7	4,661	—	72.5	14,170	—	62.5	16,682	—
OPSDC (step 100)	86.6	1,921	58.8%	69.6	9,152	35.4%	57.1	10,726	35.7%
OPSDC (step 200)	85.1	1,305	72.0%	62.1	6,902	51.3%	46.7	8,146	51.2%
<i>Qwen3-14B</i>									
Base Model	70.0	3,872	—	65.8	12,844	—	67.1	15,642	—
OPSDC (step 100)	86.1	1,686	56.5%	76.3	7,577	41.0%	61.7	10,137	35.2%
OPSDC (step 200)	86.2	1,191	69.2%	66.3	6,089	52.6%	53.8	7,496	52.1%

D Training and Implementation Details

Technical setup. All experiments are conducted on a single node equipped with eight NVIDIA H200 GPUs. Our implementation is built on top of the `ver1` library [Sheng et al. \(2025\)](#), which provides a HybridEngine for efficient actor–rollout–reference model co-location. We use PyTorch Fully Sharded Data Parallel (FSDP) for distributed training with parameter and optimizer offloading to CPU, and SGLang [Zheng et al. \(2024\)](#) for batched rollout generation. Sequence parallelism (Ulysses, degree 4) is enabled during training to handle long sequences efficiently, while tensor parallelism (degree 2) is used for inference. Mixed-precision training is performed in `bf16` at 16, and gradient checkpointing is enabled to reduce peak memory usage.

Training data. Our training data is derived from DAPO-Math-17k [Yu et al. \(2025\)](#), a deduplicated set of $\sim 17,000$ competition-level math problems. We randomly split the dataset into 80% training ($\sim 13,600$ prompts) and 20% validation ($\sim 3,400$ prompts) with a fixed seed for reproducibility across all configurations. For each problem, we construct a *student prompt* (the original question) and a *teacher prompt* (the question prepended with a conciseness instruction).

Training procedure. At each training step, the student model generates a response from the student prompt via SGLang sampling (temperature 1.0, top- p 1.0). We then perform a single gradient update minimizing the reverse KL divergence between student and teacher logit distributions over the student’s own generated tokens. All student rollouts are used for training regardless of correctness; no filtering is applied. Both teacher and student forward passes are performed for each micro-batch with chunked logit processing (chunk size 256 tokens) to bound peak GPU memory; teacher logits are progressively freed after each chunk.

Hyperparameters. Table 7 summarizes the full configuration, which is shared across all models and instruction variants.

Evaluation. We evaluate every 10 steps on three held-out math benchmarks: MATH-500 [Hendrycks et al. \(2021\)](#), AIME 2024, and AIME 2025. For each benchmark, we generate 8 responses per problem with temperature 0.6, top- $p = 0.95$, and top- $k = 20$, and report mean accuracy (fraction of correct

Parameter	Value
General	
Models	Qwen3-8B, Qwen3-14B
Loss function	Reverse KL: $\text{KL}(\pi_{\text{student}} \parallel \pi_{\text{teacher}})$
Teacher	Periodic update ($M=50$ steps)
Data	
Training prompts	$\sim 13,600$ (from DAPO-Math-17k)
Validation prompts	$\sim 3,400$
Max prompt length	1,024 tokens
Max response length	8,192 tokens (training)
Generation (student rollout)	
Inference engine	SGLang
Temperature	1.0
Top- p	1.0
Rollouts per prompt	1
Max generation tokens	9,216
Evaluation	
Temperature	0.6
Top- p	0.95
Top- k	20
Rollouts per prompt	8
Max generation tokens	30,000
Eval frequency	Every 10 steps
Training	
Optimizer	AdamW
Learning rate	1×10^{-6} (constant)
Weight decay	0.01
Gradient clipping	1.0 (max norm)
Global batch size	32
Micro-batch size per GPU	2
Epochs	1
Precision	bfloat16
Infrastructure	
GPUs	$8 \times$ NVIDIA H200
Tensor parallelism (inference)	2
Sequence parallelism (training)	Ulysses, degree 4
FSDP parameter offload	Enabled
FSDP optimizer offload	Enabled
Gradient checkpointing	Enabled

Table 7: Hyperparameters for OPSDC. The same configuration is used across all models and instruction variants.

samples averaged over problems) and average response token count. Correctness is determined by extracting the final answer and comparing against the ground truth using the symbolic and numeric equivalence checker from the `verl` library (Sheng et al., 2025).⁵

⁵https://github.com/verl-project/verl/blob/main/verl/utils/reward_score/math_dapo.py

E Alternative Teacher Parameterizations

The main paper uses a *periodic teacher update* ($\bar{\theta} \leftarrow \theta$ every M steps) that balances progressive compression with training stability. Here we discuss the design space of teacher parameterizations, from the most conservative to the most aggressive. An empirical comparison across update intervals $M \in \{1, 10, 20, 40, 50, 60\}$ is provided in Section 5.7.2.

Frozen teacher ($M = \infty$). The simplest variant fixes $\bar{\theta} = \theta_0$ for the entire training run. This provides a stable, non-shifting compression target and allows teacher prefill to be pre-computed once. However, the frozen teacher becomes an increasingly weak compression oracle as the student improves, limiting the maximum achievable compression.

Periodic teacher (our default, $M=50$). Setting a finite update interval M (Algorithm 1) enables progressive compression: after each refresh, the updated teacher, having already internalized compression from the previous round, produces even more concise traces under instruction c , providing a stronger compression signal. The discrete refresh avoids continuous co-adaptation while still allowing compression to deepen over training. Our ablation (Section 5.7.2) shows that $M \in \{40, 50, 60\}$ form a stable plateau with nearly identical training dynamics, confirming robustness to the exact interval within this range.

EMA teacher. The teacher parameters are an exponential moving average of the student, updated after each gradient step:

$$\bar{\theta} \leftarrow \alpha \bar{\theta} + (1 - \alpha) \theta, \quad \alpha \in [0.99, 0.999]. \quad (36)$$

This provides a smooth, continuous version of progressive compression. With moderate decay ($\alpha = 0.995$), it can yield additional compression beyond the frozen teacher but requires careful monitoring for collapse.

Stop-gradient concurrent teacher ($M=1$). The teacher uses the *same* parameters θ as the student, with stop-gradient during the backward pass. This provides the most aggressive progressive compression but carries the highest risk of *progressive compression collapse*: as the student becomes more concise, the teacher also becomes more concise, creating a positive feedback loop that can drive output length toward degenerate short sequences. Our ablation confirms this prediction empirically: $M=1$ causes entropy explosion and accuracy collapse (Figure 7), consistent with the moving-target instability identified by Shenfeld et al. (2026).

F Token Reduction and Accuracy over Training

Figure 8 illustrates a key practical advantage of OPSDC: **compression is achieved early and remains stable over extended training**.

For both Qwen3-8B and Qwen3-14B, the bulk of token reduction occurs within the first ~ 80 training steps, after which average token count plateaus around 3000–3500 tokens. Extending training to 200 steps yields only marginal additional compression beyond step 100, indicating that the dense, per-token KL signal enables rapid convergence to a stable compression level. This means practitioners can achieve nearly full compression benefit with modest compute budgets. Crucially, within each M -step teacher window the plateau is *stable*: token length does not continue to decrease, confirming that the fixed teacher within a window acts as a stable attractor (cf. the collapse risk with $M=1$ in Section 5.7.2).

Figure 9 confirms that accuracy improvements track compression dynamics. On MATH-500, both models show steady accuracy gains that co-occur with the token reduction in Figure 8, reinforcing the finding that compression *causes* accuracy gains rather than merely coinciding with them. For AIME 2024 and AIME 2025, the small sample sizes (30 problems each) introduce substantial evaluation variance, making step-by-step trends less reliable; nevertheless, accuracy remains stable or slightly improving throughout training, showing no evidence of an accuracy–compression trade-off. The high variance on competition benchmarks underscores the importance of using larger evaluation sets (e.g., MATH-500) for monitoring training dynamics.

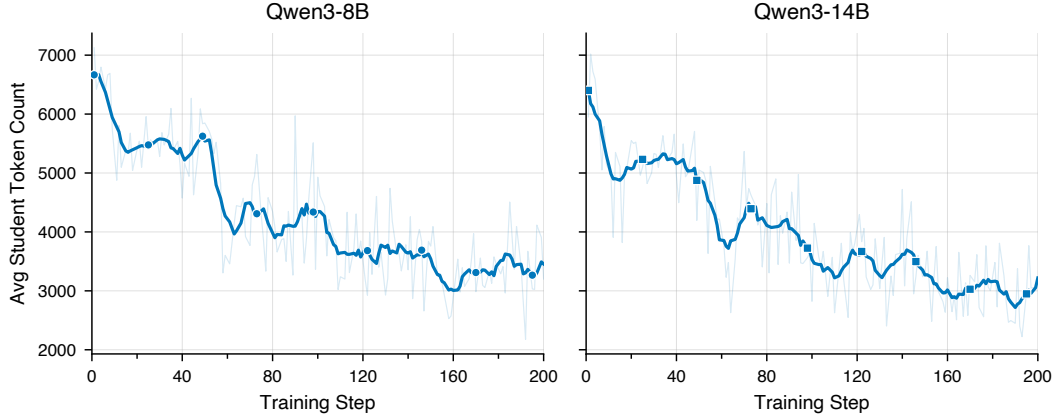


Figure 8: **Average response token count over 200 training steps** for Qwen3-8B and Qwen3-14B using the qualitative concise instruction with periodic teacher update ($M=50$). Token count decreases rapidly in the first ~ 80 steps before plateauing around 3000–3500 tokens. Further compression between steps 100 and 200 is limited, indicating that most compression is learned early and additional training yields diminishing returns.

G Effect of KL Divergence Direction in OPSDC

G.1 Background and Motivation

The OPSDC distillation loss aligns the live student p_S with the teacher p_T , a periodically frozen snapshot of the student itself. A natural question is which direction of KL divergence to use:

- **Reverse KL** (our choice): $\text{KL}(p_S \| p_T) = \sum_v p_S(v) [\log p_S(v) - \log p_T(v)]$. The gradient w.r.t. student parameters is weighted by the *student's own* distribution $p_S(v)$: the student updates only in regions where it currently generates, providing built-in self-regularization against abrupt distribution shifts.
- **Forward KL** (baseline): $\text{KL}(p_T \| p_S) = \sum_v p_T(v) [\log p_T(v) - \log p_S(v)]$. The gradient is weighted by the *teacher's* distribution $p_T(v)$, fully decoupled from the student's current state.

Forward KL is sometimes preferred in offline distillation because the teacher is a fixed, authoritative external model—its distribution is a reliable signal to track exactly. In OPSDC, however, the teacher is not external: it is a stale copy of the student, refreshed every M steps. We argue that this makes forward KL structurally ill-suited for OPSDC: because the gradient is scaled by p_T rather than p_S , every teacher refresh injects a large, unconstrained update whose magnitude is independent of how far the student has drifted. As we show below, this produces cascading instability that worsens with each successive refresh.

G.2 Experimental Setup

We compare reverse and forward KL on Qwen3-8B and Qwen3-14B with exactly the same setup as in Table 2, where teacher is updated every 50 steps. Validation pass@1 (8 samples) is measured on MATH-500, AIME 2024, and AIME 2025 every 10 steps.

G.3 Results

Figure 10 shows a stark contrast. Reverse KL rises quickly in the first 50–70 steps and holds a stable plateau through all four teacher refreshes with no visible regression. Forward KL tracks comparably in the first interval—it is even marginally ahead on MATH at step 70—but collapses within 10 steps of every subsequent refresh before partially recovering. The saw-tooth deepens with each cycle: on Qwen3-14B the trough relative to reverse KL grows from $\sim 15\%$ on AIME 2024 after the step-100

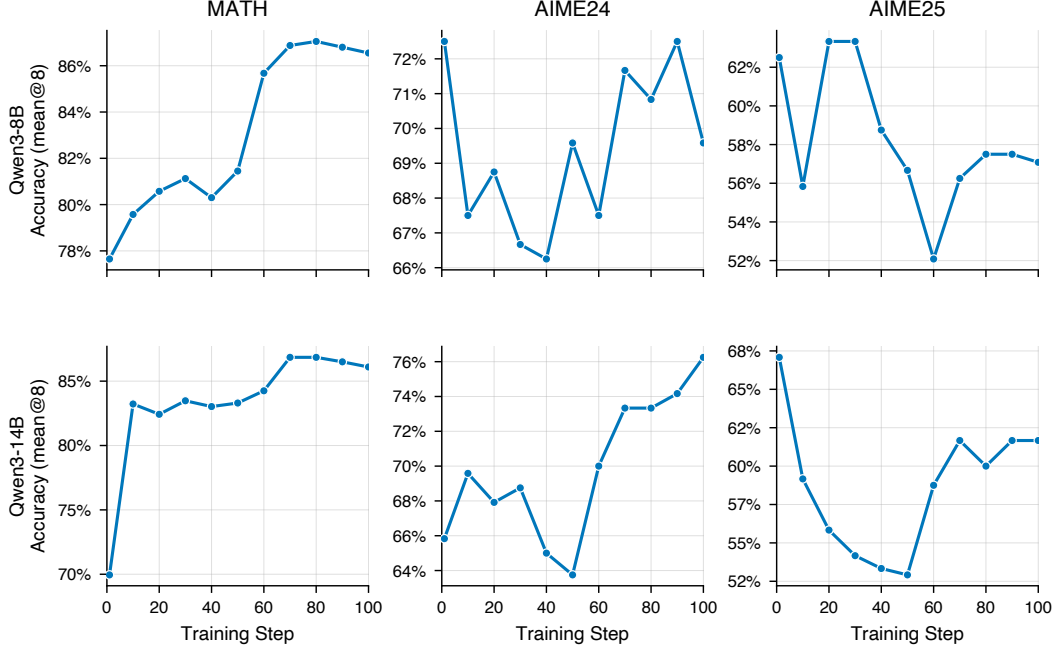


Figure 9: **Validation accuracy (mean@8) over training steps** for Qwen3-8B and Qwen3-14B using the qualitative concise instruction with periodic teacher update ($M=50$), evaluated on MATH-500, AIME 2024, and AIME 2025. MATH-500 accuracy improves steadily for both models, rising from $\sim 78\%$ to $\sim 87\%$ (8B) and $\sim 70\%$ to $\sim 87\%$ (14B). AIME 2024 and AIME 2025 results exhibit substantially larger variance due to their small sample sizes (30 problems each), though the overall trend remains stable or slightly improving.

refresh, to $> 19\%$ after step 150, reaching a $> 23\%$ gap by step 190. The same pattern is visible but milder on Qwen3-8B.

Figure 11 reveals a parallel instability: forward KL compresses responses more aggressively, with length drops synchronized to the same teacher-update boundaries. On Qwen3-14B the gap widens to ~ 500 tokens by step 190 (1,229 vs. 1,766, a 30% shortfall). On hard reasoning benchmarks like AIME, this truncation of thinking chains is both a symptom and a driver of the accuracy degradation.

G.4 Mechanism and Conclusion

The instability follows directly from the gradient structure. Under forward KL, $\nabla_{\log p_S} \text{KL}(p_T \| p_S) = -p_T(v)$: updates are scaled by the teacher’s confidence regardless of the student’s current state. Each refreshed teacher is slightly more length-compressed than the last, so the distributional shock at each refresh grows, explaining the escalating saw-tooth amplitude. Under reverse KL, the gradient takes the form $\nabla_{\theta} \text{KL}(p_S \| p_T) = \mathbb{E}_{v \sim p_S} [\nabla_{\theta} \log p_S(v) (\log p_S(v) - \log p_T(v) + 1)]$, so updates are explicitly weighted by the student’s own distribution p_S ; this provides natural self-regularization. Because the student already covers the teacher’s high-probability modes, each refresh requires only a small incremental adjustment.

Reverse KL is the appropriate divergence for iterative on-policy self-distillation. All OPSDC experiments in this paper use reverse KL accordingly.

H Qualitative Examples

Figure 12 provides side-by-side comparisons of *full model outputs* from the base Qwen3-8B model and the OPSDC-trained model on three MATH-500 problems of increasing difficulty. Each output consists of hidden reasoning (inside `<think>...</think>` tags) followed by the visible answer presented to the user. The base model exhibits two distinct sources of token overhead:

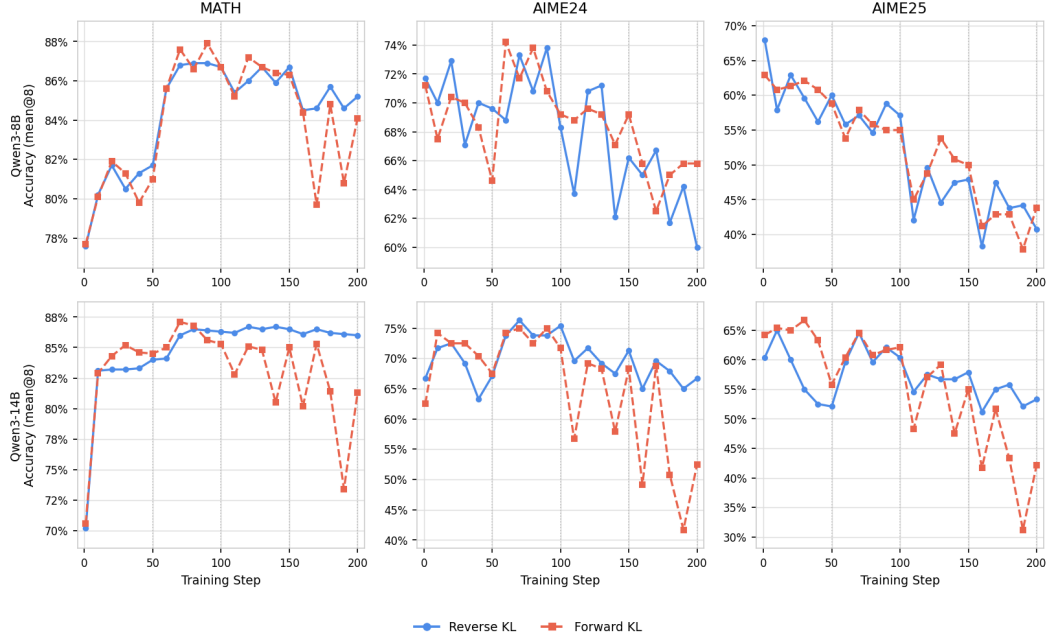


Figure 10: Validation accuracy of reverse KL (solid blue) and forward KL (dashed coral) on Qwen3-8B (top) and Qwen3-14B (bottom). Dotted verticals mark teacher-update steps. Reverse KL holds a stable plateau; forward KL exhibits a saw-tooth that deepens with each refresh.

- **Redundancy within reasoning:** Self-doubt (“Wait,” “Let me check...”), re-derivation via alternative methods, and numerical verification of already-established results inside `<think>`.
- **Redundancy between reasoning and visible answer:** The base model produces a fully formatted, step-by-step solution *after* `</think>` that essentially repeats the entire derivation from the hidden reasoning block.

OPSDC eliminates both: it compresses the hidden reasoning to core logical steps and produces only the final answer as visible output. The three examples also illustrate difficulty-adaptive compression (Section 3.3):

- **Easy problem (84% reduction):** The algebra word problem admits a short, direct derivation. The base model re-derives via alternative methods and verifies numerically inside `<think>`, then writes a full formatted solution after `</think>`. OPSDC retains only the single derivation and outputs the answer directly.
- **Moderate problem (56% reduction):** The number theory problem requires a genuine insight ($10^k \equiv 10 \pmod{18}$). The base model discovers this but repeatedly verifies it, cross-checks via CRT, and writes a formatted step-by-step solution after `</think>`. OPSDC discovers the same insight, applies it once with a brief CRT check, and outputs only the answer.
- **Hard problem (52% reduction):** The product-simplification problem requires the Sophie Germain identity and a telescoping argument. The base model arrives at the correct answer but only after extensive numerical exploration, repeated algebraic verification, and an attempt to factor the final result; after `</think>`, it reproduces the full four-step derivation. OPSDC applies the identity immediately, identifies the telescoping pattern, and outputs only the answer.

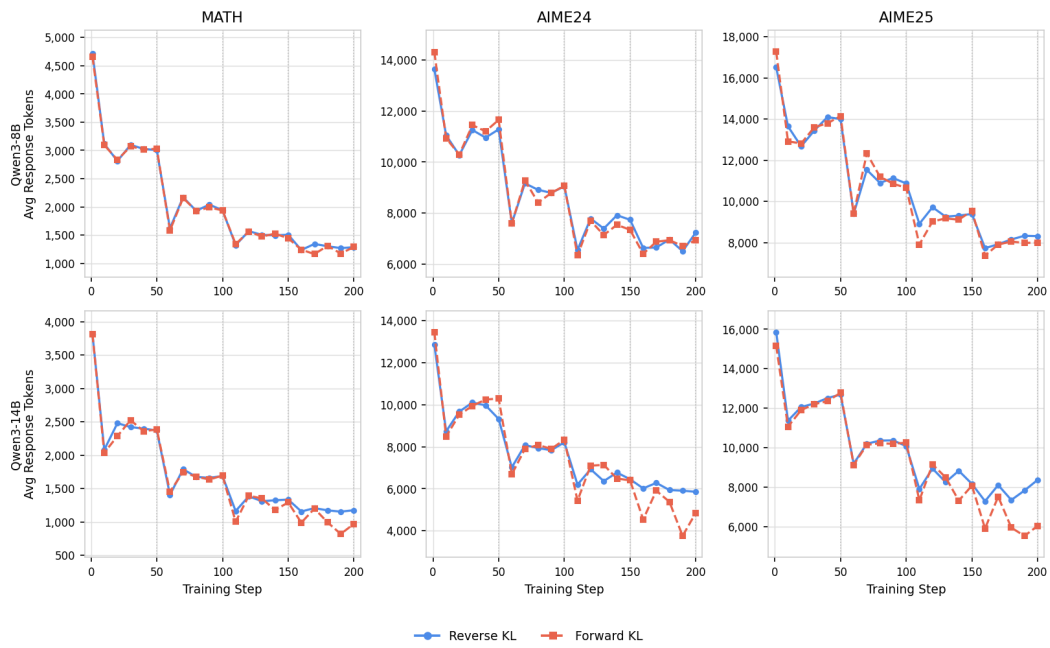


Figure 11: Mean response length on each validation set. Forward KL compresses responses more aggressively, with steeper drops synchronized with each teacher-update boundary.

Problem 1 (Algebra): Ten treeks weigh as much as three squigs and one goolee. Two treeks and one goolee are equal in weight to one squig. The combined weight of how many treeks equals one squig?

Base model (1,354 tokens)

```
<think>
... I need to figure out how many treeks weigh the same as one
squig...
 $10T = 3S + G$  (first equation)
 $2T + G = S$  (second equation)
From the second equation:  $G = S - 2T$ . Substitute:
 $10T = 3S + (S - 2T) \Rightarrow 12T = 4S \Rightarrow 3T = S$ .
Wait, so S equals 3T? ... Let me check my steps again...
Alternatively, let's use substitution. Let me express everything in
terms of T...  $10T = 6T + 4G \Rightarrow T = G$ . Which again gives  $S =$ 
 $3T$ . So same result.
So, if  $T = G$ , then...  $9T = 3S \Rightarrow 3T = S$ . Yep, same thing.
Wait, but let me check with actual numbers... Suppose a treek
is 1 kg. Then squig is 3 kg, goolee is 1 kg. Check first statement:
 $10 = 10$ . Correct. Second:  $3 = 3$ . Correct.
</think>
```

To determine how many treeks weigh the same as one squig, we
are given...

Step 1: Express G in terms of S and T : $G = S - 2T$.
Step 2: Substituting into $10T = 3S + G$... $10T = 4S - 2T$.
Step 3: Add $2T$... $12T = 4S \Rightarrow 3T = S$.
This means that **one squig weighs the same as 3 treeks**.
Answer: **3** ✓

OPSDC (221 tokens, 84% reduction)

```
<think>
The problem gives two equations. 10 treeks = 3 squigs + 1 goolee.
2 treeks + 1 goolee = 1 squig.
Equation 1:  $10T = 3S + G$ 
Equation 2:  $2T + G = S$ 
From Equation 2, solve for  $G$ :  $G = S - 2T$ . Substitute into Equation
1:
 $10T = 3S + (S - 2T)$ 
 $10T = 4S - 2T$ 
 $12T = 4S$ 
Divide both sides by 4:  $3T = S$ 
So, 1 squig equals 3 treeks. Therefore, the answer is 3.
</think>
```

Answer: **3** ✓

Problem 2 (Number Theory): What integer n satisfies $0 \leq n < 18$ and $n \equiv -11213141 \pmod{18}$?

Base model (3,979 tokens)

```
<think>
... I need to find  $n \equiv -11213141 \pmod{18}$ ...
Since 18 is  $2 \times 9$ , maybe I can use the Chinese Remainder
Theorem? Wait, maybe that's overcomplicating.
 $10^k \equiv 10 \pmod{18}$  for  $k \geq 1$ . Let me check:  $10^1 = 10$ ,
 $10^2 = 100 \equiv 10$ ,  $10^3 \equiv 10$ ... Yes.
Digits: 1,1,2,1,3,1,4,1.  $1 + 10 \times (4+1+3+1+2+1+1) =$ 
 $131$ .  $131 \pmod{18} = 5$ . Let me verify:  $18 \times 7 = 126$ ,
 $131 - 126 = 5$ . Yes.
So  $-11213141 \equiv -5 \equiv 13 \pmod{18}$ .
Let me verify by another method... CRT: mod 2 gives 1, mod 9:
digit sum 14  $\equiv 5$ . Numbers 0-17 that are 4 mod 9: 4, 13. 13 is
odd. So  $n = 13$ .
All methods point to 13.
</think>
```

Step 1: We compute $11213141 \pmod{18}$, then negate.
Step 2: Using $10^k \equiv 10 \pmod{18}$... $(1+1+2+1+3+1+4) \times 10 + 1 = 131 \equiv 5 \pmod{18}$.
Step 3: Convert to positive: $-5 + 18 = 13$.
Answer: **13** ✓

OPSDC (1,758 tokens, 56% reduction)

```
<think>
I need to find  $n \equiv -11213141 \pmod{18}$ ...
 $10^k \equiv 10 \pmod{18}$  for  $k \geq 1$ . So any number mod 18 is:
 $d_0 + 10 \cdot (d_1 + d_2 + \dots + d_n)$ .
Digits of 11213141:
 $d_7=1, d_6=1, d_5=2, d_4=1, d_3=3, d_2=1, d_1=4, d_0=1$ .
 $d_0 + 10 \cdot (4 + 1 + 3 + 1 + 2 + 1 + 1) =$ 
 $= 1 + 10 \cdot 13 = 131$ 
 $131 \pmod{18} = 5$  ( $18 \times 7 = 126$ )
So  $-11213141 \equiv -5 \equiv 13 \pmod{18}$ .  $n = 13$ .
CRT verification... mod 2  $\rightarrow 1$ , mod 9  $\rightarrow 5$ . Gives  $n = 13$ .
Same result.
</think>
```

Answer: **13** ✓

Figure 12: **Full model outputs: base Qwen3-8B vs. OPSDC** on three MATH-500 problems of increasing difficulty. Each output consists of hidden reasoning (between `<think>` and `</think>`) followed by the visible answer (below the gray rule). **Blue text** highlights redundancy: within reasoning (self-doubt, re-derivation, verification) and in the visible answer (the base model repeats the full derivation as a formatted step-by-step solution, while OPSDC outputs only the answer). **Top:** an easy algebra problem (84% reduction). **Bottom:** a number theory problem (56% reduction). Problem 3 continues in Figure 13.

Problem 3 (Algebra/Number Theory): Let n be a positive integer. Simplify

$$\frac{(2^4 + \frac{1}{4})(4^4 + \frac{1}{4}) \cdots [(2n)^4 + \frac{1}{4}]}{(1^4 + \frac{1}{4})(3^4 + \frac{1}{4}) \cdots [(2n-1)^4 + \frac{1}{4}]}$$

Base model (6,639 tokens)

<think>

...I need to simplify this big fraction. . .

Let me try to compute a few terms for small n . . . For $n=1$: $16.25/1.25 = 13$. Hmm, 13. Let me try $n=2$. $16.25 \times 256.25/(1.25 \times 81.25) = 13 \times 41/13 = 41$.

Maybe there's a telescoping product? Let me think about how to factor $x^4 + 1/4$. . . Sophie Germain identity: $a^4 + 4b^4 = (a^2 + 2ab + 2b^2)(a^2 - 2ab + 2b^2)$. Let me check...compute it properly...

With $b=1/2$: $x^4 + 1/4 = (x^2 + x + 1/2)(x^2 - x + 1/2)$.

Numerator factor: $A_k = 4k^2 + 2k + 1/2$, $B_k = 4k^2 - 2k + 1/2$.

Denominator factor: $C_k = 4k^2 - 2k + 1/2 = B_k$. So B_k cancels!

$D_k = 4k^2 - 6k + 5/2 = A_{k-1}$. Telescopes:

$\prod A_k / \prod A_{k-1} = A_n / A_0$.

$A_n / A_0 = (4n^2 + 2n + 1/2) / (1/2) = 8n^2 + 4n + 1$.

Let me check: $n=1$: $8+4+1=13$. Correct. $n=2$: $32+8+1=41$. Correct.

Let me see if it factors. Discriminant is $16 - 32 = -16$, so it doesn't factor.

</think>

To simplify the given expression, we begin by analyzing the structure...

Step 1: Use the Sophie Germain Identity: $x^4 + 1/4 = (x^2 + x + 1/2)(x^2 - x + 1/2)$.

Step 2: Apply the Factorization to numerator and denominator terms. . . the term $\prod (4m^2 - 2m + 1/2)$ cancels.

Step 3: Observe the Telescoping Pattern: $D_k = A_{k-1}$... only A_n / A_0 remains.

Step 4: Final Simplification: $(4n^2 + 2n + 1/2) / (1/2) = 8n^2 + 4n + 1$.

Answer: $8n^2 + 4n + 1$ ✓

OPSDC (3,202 tokens, 52% reduction)

<think>

I need to simplify this big fraction. . . I remember that $a^4 + 4b^4$ can be factored using the Sophie Germain identity. With $b = 1/2$: $x^4 + 1/4 = (x^2 + x + 1/2)(x^2 - x + 1/2)$.

Numerator: $A_k = 4k^2 + 2k + 1/2$, $B_k = 4k^2 - 2k + 1/2$.

Denominator: first factor is $4k^2 - 2k + 1/2 = B_k$, second is $D_k = 4k^2 - 6k + 5/2$.

Since $B_k = C_k$, those cancel: ratio is $\prod A_k / \prod D_k$.

$A_{k-1} = 4(k-1)^2 + 2(k-1) + 1/2 = 4k^2 - 6k + 5/2 = D_k$.

Telescopes: $A_n / A_0 = (4n^2 + 2n + 1/2) / (1/2) = 8n^2 + 4n + 1$.

Check: $n=1$: 13. $n=2$: 41. Correct.

</think>

Answer: $8n^2 + 4n + 1$ ✓

Figure 13: **Full model outputs (continued from Figure 12): a hard product-simplification problem** requiring the Sophie Germain identity and a telescoping argument (52% reduction). The base model arrives at the answer only after extensive numerical exploration, repeated verification, and an attempt to factor the result; after </think>, it reproduces the full four-step derivation. OPSDC applies the identity immediately, identifies the telescoping pattern, and outputs only the answer. The harder problem retains more reasoning than the easy (84%) and moderate (56%) cases in Figure 12, illustrating difficulty-adaptive compression (Section 3.3).